Universidad Autónoma de Entre Ríos — Facultad de Ciencia y Tecnología
Licenciatura en Sistemas de Información — Sistemas Operativos / Sistemas Distribuidos

Relojería

Caso práctico: Sincronización de relojes en Sistemas Distribuidos

Alumno	Julián Olivera
Docente	Ing. Mg. Ulises Rapallini
Repositorio	https://github.com/julianAO2002/relojeria
Fecha	11/11/2026

Descripción del sistema

Sincronización de relojes en sistemas distribuidos con **C#, ASP.NET Core y Docker**. Los tres algoritmos del caso práctico funcionando como servicios independientes:

- **Cristian**: un servidor de tiempo y tres clientes que estiman la hora con `hora del servidor + RTT/2`.
- **Berkeley**: un coordinador que promedia los relojes de cuatro servidores (él incluido) y le manda a cada uno su ajuste.
- **Lamport**: `OrdersService`, `InventoryService` y `PaymentsService` intercambian mensajes HTTP con su reloj lógico en un header.

Caso práctico de sincronización — Sistemas Operativos / Sistemas Distribuidos (FCyT – UADER). Autor: Julián Olivera.

Contenido

Ruta	Qué hay
`src/Clocks/`	Biblioteca compartida: reloj simulado, lógica pura de Cristian y Berkeley, relojes de Lamport y vectoriales, registro de eventos.
`src/Cristian.TimeServer/`	Web API: `GET /api/time` (con latencia de red simulable).
`src/Cristian.Client/`	Cliente: mide T0, T1 y RTT, estima la hora, ajusta su reloj de aplicación.
`src/Berkeley.Node/`	Web API de cada nodo: `GET /api/clock`, `POST /api/clock/adjust`.
`src/Berkeley.Coordinator/`	Coordinador: sondea, promedia y envía los ajustes cada `INTERVAL_SECONDS`.
`src/Lamport.Orders/`, `src/Lamport.Inventory/`, `src/Lamport.Payments/`	Los tres servicios del ejercicio 3; `GET /api/timeline` en Orders junta los eventos de los tres.
`tests/Clocks.Tests/`	10 tests unitarios (xUnit) de la lógica de los algoritmos.
`Dockerfile`	Un único Dockerfile multi-etapa; el proyecto se elige con `--build-arg PROJECT=<nombre>`.
`docker-compose.yml`	Un *profile* por ejercicio: `cristian`, `berkeley`, `lamport`.
`scripts/`	Un script por ejercicio que levanta el escenario, provoca las fallas y guarda el registro en `docs/salidas/`.
`docs/salidas/`	Registros de ejecución reales.

Cómo se simulan los relojes

Todos los contenedores de Docker comparten el reloj del kernel del host, así que no se puede desfasar el reloj del sistema operativo de cada uno (y el enunciado pide justamente **no** tocarlo). Cada aplicación tiene un **reloj de aplicación** (`SimulatedClock`):

```
hora = hora real UTC + desfase inicial + deriva + correcciones aplicadas por el algoritmo
```

Variable	Significado
`CLOCK_OFFSET_SECONDS`	Desfase inicial (por ejemplo `8` para el servidor B del enunciado).
`CLOCK_DRIFT_PPM`	Deriva en partes por millón: `1000` = el reloj adelanta 1 ms por segundo.
`SCENARIO_START_UTC`	Instante real que se muestra como `10:00:00` (lo fijan los scripts), para que los registros coincidan con las horas del enunciado. Vacío = hora UTC real.

Los algoritmos solo usan `Adjust(...)` sobre ese reloj. Como la hora real es común a todos los contenedores, los registros pueden mostrar el **error real** de cada estimación, algo que en un sistema real no se puede medir. Ese valor es solo para el análisis: el algoritmo nunca lo lee.

Ejecución

Requisitos: Docker con Compose v2 y Bash (Git Bash en Windows). Para los tests, .NET SDK 8 o superior.

```
dotnet test tests/Clocks.Tests              # 10 tests de la logica

bash scripts/ej1_cristian.sh                 # -> docs/salidas/ej1_cristian.txt
bash scripts/ej2_berkeley.sh                 # -> docs/salidas/ej2_berkeley.txt (7 escenarios, ~3 min)
bash scripts/ej3_lamport.sh                  # -> docs/salidas/ej3_lamport.txt

# O a mano, por ejercicio:
docker compose --profile cristian up --build
docker compose --profile berkeley up --build
docker compose --profile lamport up --build -d
curl -X POST localhost:8090/api/orders -H "content-type: application/json" -d '{"productId":"P001","quantity":
2,"amount":150}'
curl localhost:8090/api/timeline
```

Respuestas

Preguntas teóricas

1. ¿Cuál es la diferencia entre un reloj físico y un reloj lógico? Un **reloj físico** mide el paso del tiempo real: en una computadora es un oscilador de cuarzo que genera interrupciones a frecuencia fija, que el sistema operativo cuenta para obtener la hora. Sincronizarlo significa acercar su valor a una referencia (UTC) o a los otros relojes. Un **reloj lógico** no mide tiempo: es un contador que numera los eventos de modo que respeten la causalidad (si *a* causó *b*, el número de *a* es menor). No dice qué hora es ni cuánto tiempo pasó, solo qué ocurrió antes de qué.

	Reloj físico	Reloj lógico
Representa	La hora real (aproximada)	El orden de los eventos
Base	Hardware (cuarzo)	Software (contador)
Se sincroniza con	UTC, NTP, Cristian, Berkeley	Las marcas de tiempo de los mensajes
Problema principal	La deriva	No mide duraciones; solo captura causalidad
Uso	Auditoría, certificados, timeouts, logs con fecha	Orden de mensajes, exclusión mutua distribuida, depuración

2. ¿Qué es la deriva de un reloj y cómo afecta a los sistemas distribuidos? La **deriva** (*drift*) es la diferencia de **velocidad** entre un reloj y el tiempo real (o entre dos relojes). Ningún cristal oscila exactamente a

su frecuencia nominal: depende de la fabricación, la temperatura y el envejecimiento. Si $C(t)$ es el valor del reloj en el instante real t, un reloj perfecto cumple $dC/dt = 1$, y uno real cumple $1 - \rho \le dC/dt \le 1 + \rho$, donde ρ es la tasa máxima de deriva (en cuarzo típico ≈ 10^{-6}, unos 31 s por año). Hay que distinguirla del **desfase** (*offset* o *skew*), que es la diferencia de *valor* en un instante: la deriva es lo que hace crecer el desfase.

En un sistema distribuido, aunque todos los relojes se pongan en hora, con el tiempo se separan: dos relojes que derivan en sentidos opuestos difieren hasta 2ρΔt después de Δt. Eso rompe todo lo que compara marcas de tiempo de distintas máquinas: logs que no se pueden ordenar, un archivo que parece más viejo que su fuente (`make` no recompila), certificados o tokens "todavía no válidos" o "vencidos", timeouts y alquileres (*leases*) mal calculados, o el criterio "gana la última escritura" en bases replicadas que pisa el dato equivocado. Por eso la sincronización tiene que repetirse periódicamente, al menos cada δ/2ρ segundos para no superar un desfase δ. En la práctica se ve en el ejercicio 1: el servidor B adelanta 5000 ppm y hay que corregirlo 20 ms cada 4 s; con el servidor de tiempo caído pasó unos 10 s sin sincronizar y su error llegó a +53 ms.

3. ¿Qué es el Tiempo Atómico Internacional (TAI) y cuál es su relación con UTC? El **TAI** es la escala de tiempo calculada por la Oficina Internacional de Pesas y Medidas (BIPM, París) promediando cientos de relojes atómicos de cesio 133 de laboratorios de todo el mundo. Desde 1967 el segundo se define como 9.192.631.770 períodos de la radiación de transición del cesio 133. Es extremadamente estable, pero no sigue la rotación de la Tierra, que se va frenando: el día solar se alarga y el TAI se adelanta respecto del tiempo astronómico. El **UTC** (Tiempo Universal Coordinado) es el TAI corregido con **segundos intercalares** (*leap seconds*) que se agregan cuando la diferencia con el tiempo solar se acerca a 0,9 s. Por eso UTC avanza al ritmo del TAI pero va "atrasado" un número entero de segundos (37 s desde 2017). UTC es la base de la hora civil y se distribuye por radio, GPS e Internet (NTP).

4. ¿Por qué resulta difícil establecer un orden global de eventos utilizando únicamente relojes físicos? Porque no existe un reloj global: cada máquina tiene el suyo, con desfase y deriva distintos, y aun sincronizados quedan diferencias del orden de la precisión de la sincronización (milisegundos en una LAN con NTP, más en Internet). Si dos eventos de máquinas distintas ocurren más cerca que esa precisión, sus marcas de tiempo pueden quedar invertidas. Peor aún: un mensaje puede aparecer **recibido antes de ser enviado**. En el ejercicio 3, con el reloj de `InventoryService` 3 s atrasado, el registro dice que Inventory recibió el pedido **2,861 s antes** de que Orders lo enviara, y que Orders recibió la confirmación de pago **4,990 s antes** de que Payments la mandara. Ordenar por hora física da 93 pares de eventos en orden imposible. Además, la latencia de red es variable y desconocida, así que ni siquiera se puede saber con exactitud cuándo se leyó un reloj remoto.

5. ¿Cómo funciona el algoritmo de Cristian? Explique sus ventajas y limitaciones. Hay un **servidor de tiempo** (con una buena referencia, por ejemplo un receptor de UTC). Cada cliente:

1. anota su hora local **T0** y le pide la hora al servidor;
2. el servidor responde con su hora **Ts**;
3. el cliente anota **T1** al recibir la respuesta y calcula **RTT = T1 − T0**;
4. estima la hora actual como **Ts + RTT/2**, suponiendo que la ida y la vuelta tardaron lo mismo;
5. ajusta su reloj. La precisión de la estimación es **±RTT/2**, porque el servidor leyó su reloj en algún momento dentro de ese intervalo.

Para mejorar el resultado se toman varias muestras y se usa la de menor RTT (la más precisa) o se descartan las de RTT excesivo, que es lo que hace el cliente implementado. El tiempo de procesamiento del servidor, si se conoce, puede restarse del RTT. Además, el reloj no debería ir hacia atrás: si hay que atrasarlo, conviene frenarlo de a poco (*slewing*).

- **Ventajas**: muy simple, pocos mensajes (dos por sincronización), buena precisión en redes con RTT chico y simétrico (en Docker, error real de 0,1 ms), y sincroniza con una referencia externa (hora real).

- **Limitaciones**: el servidor es un **punto único de falla** y un posible cuello de botella; supone latencia **simétrica** (con 50 ms de ida y 450 ms de vuelta el error fue de −201 ms); con RTT grande la precisión empeora; si el servidor da una hora errónea o es malicioso, todos los clientes la copian; y no tolera fallas bizantinas.

6. ¿Cómo funciona el algoritmo de Berkeley? ¿Qué función cumple el nodo coordinador? No supone que alguien tenga la hora correcta: busca que **todos coincidan entre sí**. El **coordinador** (*time daemon*):

1. le pide la hora a cada nodo (compensando la latencia con RTT/2, como Cristian);
2. calcula la diferencia de cada uno con su propio reloj;
3. promedia las diferencias **incluyendo la suya** (0), opcionalmente descartando los relojes muy desviados (promedio tolerante a fallas);
4. calcula para cada nodo el ajuste = promedio − diferencia y
5. le envía a cada uno **el ajuste**, no la hora: un valor relativo no se desactualiza por lo que tarde en llegar el mensaje.
6. Repite periódicamente.

El coordinador **organiza** la sincronización: sondea, calcula el promedio y distribuye las correcciones. No es una fuente de hora “verdadera”: él también se ajusta al promedio. Se lo elige con algún algoritmo de elección (por ejemplo el del matón, *bully*) y, si falla, se elige otro.

7. ¿Qué diferencias existen entre los algoritmos de Cristian y Berkeley?

	Cristian	**Berkeley**
Modelo	Cliente–servidor: el cliente *pide* (pull)	Coordinador activo: *sondea* a los nodos (push de ajustes)
Referencia	El servidor tiene la hora correcta (por ejemplo UTC)	Nadie tiene la hora correcta; se busca el acuerdo
Resultado	Todos copian la hora del servidor (sincronización externa)	Todos convergen al promedio (sincronización interna); puede alejarse de UTC
¿Quién ajusta?	Solo los clientes	Todos, incluido el coordinador
Qué se envía	La hora del servidor	El ajuste relativo de cada nodo
Relojes desviados	Si el servidor está mal, todos quedan mal	El promedio tolerante a fallas descarta los relojes atípicos
Uso típico	Clientes con acceso a un servidor UTC (NTP se basa en esta idea)	Redes aisladas sin acceso a UTC (el original era para Berkeley UNIX)

8. ¿Qué es la relación happens-before definida por Lamport? Es la relación de orden “ocurre antes” (**a → b**) que se puede determinar **sin relojes**, solo observando el sistema:

1. si *a* y *b* son eventos del **mismo proceso** y *a* ocurre antes que *b*, entonces $a \rightarrow b$;
2. si *a* es el **envío** de un mensaje y *b* su **recepción**, entonces $a \rightarrow b$ (no se puede recibir algo antes de enviarlo);
3. es **transitiva**: si $a \rightarrow b$ y $b \rightarrow c$, entonces $a \rightarrow c$.

Captura la **causalidad potencial**: si $a \rightarrow b$, *a* pudo haber influido en *b*. Es un orden **parcial**: hay pares de eventos que no están relacionados. Lamport parte de la observación de que, en la mayoría de los casos, los procesos no necesitan saber qué hora es, sino ponerse de acuerdo en el orden de los eventos que se afectan entre sí.

9. ¿Cómo funcionan los relojes lógicos de Lamport? Cada proceso *Pi* tiene un contador *Ci* que empieza en 0:

1. **antes de cada evento** (interno o de envío): *Ci = Ci + 1*;
2. **al enviar** un mensaje, le adjunta el valor *Ci* (aquí, en el header HTTP `X-Lamport-Clock`);
3. **al recibir** un mensaje con marca *t*: *Ci = max(Ci, t) + 1*, y ese valor es el del evento de recepción.

Así se cumple la **condición de reloj**: si $a \rightarrow b$ entonces $C(a) < C(b)$. Para obtener un **orden total** (necesario por ejemplo en exclusión mutua distribuida o en multicast totalmente ordenado) se desempatan los valores iguales con el identificador del proceso: $(C(a), i) < (C(b), j)$. En el ejercicio 3: Orders envía con L=2, Inventory (que estaba en 0) recibe y pasa a 3, y Payments (que ya estaba en 4 por pagos anteriores) recibe L=7 y pasa a 8.

10. ¿Qué significa que dos eventos sean concurrentes? Que **ninguno ocurre antes que el otro**: no se cumple $a \rightarrow b$ ni $b \rightarrow a$ (se escribe $a \parallel b$). No hay ninguna cadena de eventos locales y mensajes que los conecte, así que ninguno pudo influir en el otro. No significa que ocurrieran "al mismo tiempo" en el reloj de pared, sino que es **imposible y además irrelevante** decidir cuál fue primero. En el ejercicio 3, "Pedido recibido ORD-001" en Orders y los pagos de otros pedidos que procesó Payments son concurrentes: Payments no supo nada de ese pedido hasta que recibió el mensaje L=7.

11. ¿Qué diferencias existen entre sincronizar relojes físicos y establecer un orden lógico de eventos?

	Sincronización de relojes físicos	**Orden lógico de eventos**
Objetivo	Que los relojes marquen (casi) la misma hora, y opcionalmente la hora real	Que todos coincidan en el orden causal de los eventos
Mecanismos	Cristian, Berkeley, NTP, PTP	Relojes de Lamport, relojes vectoriales
Precisión	Aproximada: siempre queda un error (±RTT/2, deriva)	Exacta respecto de la causalidad
Costo	Mensajes periódicos de sincronización	Unos bytes extra en cada mensaje de la aplicación
Qué responde	"¿Qué hora es?" y "¿cuánto duró?"	"¿Qué pasó antes?"
Limitación	No garantiza el orden de eventos más cercanos que la precisión	No dice nada de la hora real ni de las duraciones

Se complementan: el ejercicio 3 muestra que un error de unos pocos segundos en el reloj físico invierte el orden de los mensajes, mientras que Lamport los ordena bien sin importar ese error.

12. ¿Cómo funciona NTP y qué problemas permite resolver? NTP (*Network Time Protocol*, RFC 5905) sincroniza relojes con UTC a través de Internet. Usa una jerarquía de servidores por **estratos**: el estrato 0 son las fuentes de referencia (relojes atómicos, GPS), el estrato 1 los servidores conectados directamente a ellas, el 2 los que se sincronizan con el 1, etc. Cada intercambio registra cuatro marcas de tiempo: *T1* (envío del cliente), *T2* (recepción en el servidor), *T3* (envío del servidor) y *T4* (recepción en el cliente), y con ellas calcula:

```
desfase  θ = ((T2 − T1) + (T3 − T4)) / 2
demora   δ = (T4 − T1) − (T3 − T2)
```

Es la idea de Cristian, pero descontando el tiempo de proceso del servidor. NTP repite el intercambio, se queda con las muestras de menor demora, filtra y combina varios servidores para descartar los erróneos, y corrige el reloj de a poco (*slewing*) en vez de saltar, para que nunca vaya hacia atrás. Logra unos pocos milisegundos de precisión en Internet y menos de uno en una LAN.

Resuelve: la **deriva** de los relojes de cuarzo (corrección continua), la **latencia variable** de la red (estimación de demora y filtrado), la **escalabilidad** (jerarquía de estratos y redundancia) y la **tolerancia a fallas** (varios servidores, descarte de los que no coinciden, autenticación opcional). Es lo que mantiene en hora prácticamente todas las computadoras y servidores conectados.

Ejercicio 1 — Algoritmo de Cristian

Implementación. `Cristian.TimeServer` es una Web API con `GET /api/time` que devuelve la hora de su reloj (10:00:02 al inicio). Para simular la red acepta `requestDelayMs` (espera antes de leer el reloj, el viaje de ida) y `responseDelayMs` (espera después, el viaje de vuelta). Tres contenedores del cliente (`Cristian.Client`) representan a los servidores A (10:00:00), B (10:00:08, con una deriva de 5000 ppm) y C (09:59:55). Cada sincronización hace los siete pasos del enunciado:

```
var t0 = clock.Now;                                    // 2. instante de envio
var stopwatch = Stopwatch.StartNew();
var reply = await http.GetFromJsonAsync<TimeReply>("/api/time?...");   // 1. consulta REST
stopwatch.Stop();
var t1 = clock.Now;                                    // 3. instante de recepcion
var sample = new CristianSample(t0, t1, reply.ServerTime, stopwatch.Elapsed);   // 4. RTT
// 5. EstimatedNow = ServerTime + Rtt / 2 ; Offset = EstimatedNow - LocalReceive
var best = Cristian.BestSample(samples, maxRtt);       // la de menor RTT, descartando RTT > 1000 ms
clock.Adjust(best.Offset);                             // 6. ajusta el reloj de aplicacion
Log($"antes {before} | despues {after} | ajuste {best.Offset} | error real ...");   // 7.
```

El RTT se mide con `Stopwatch` (un reloj monotónico), no restando horas del reloj que se está ajustando.

Resultados (`docs/salidas/ej1_cristian.txt`):

```
Servidor A (10:00:00) - fase 'normal'
  muestra 1: T0=10:00:00.040 T1=10:00:00.125 servidor=10:00:02.055 RTT=85.3 ms
  muestra 2: T0=10:00:00.126 T1=10:00:00.128 servidor=10:00:02.127 RTT=1.8 ms
  muestra 3: T0=10:00:00.128 T1=10:00:00.129 servidor=10:00:02.128 RTT=1.0 ms
  mejor muestra RTT=1.0 ms -> hora estimada = servidor + RTT/2 (precision +/-0.5 ms)
  antes 10:00:00.135 | despues 10:00:02.135 | ajuste +2.000 s | error real -0.1 ms

Servidor B (10:00:08):  antes 10:00:08.215 | despues 10:00:02.214 | ajuste -6.001 s | error real -0.1 ms
Servidor C (09:59:55):  antes 09:59:55.196 | despues 10:00:02.196 | ajuste +7.000 s | error real -0.2 ms
```

Servidor	Hora inicial	Ajuste	Error real después
A	10:00:00	+2,000 s	−0,1 ms
B	10:00:08	−6,001 s	−0,1 ms
C	09:59:55	+7,000 s	−0,2 ms

El servidor A además repite la sincronización en distintas condiciones de red:

Fase de A	RTT de la mejor muestra	Precisión (±RTT/2)	Error real
Red normal	1,0 ms	±0,5 ms	−0,1 ms
Latencia simétrica (200 + 200 ms)	401,7 ms	±200,9 ms	−0,2 ms
Latencia asimétrica (50 + 450 ms)	504,2 ms	±252,1 ms	**−201,3 ms**
RTT muy alto (1500 + 1500 ms)	3001,8 ms → las 3 muestras se descartan	—	se mantiene el reloj (−201,3 ms)
Resincronización siguiente (red normal)	0,6 ms	±0,3 ms	−0,0 ms

Con deriva, el servidor B se va 20 ms cada 4 s y cada resincronización lo corrige (`ajuste -0.020 s`).

Falla del servidor de tiempo. El script detuvo `time-server` durante unos 9 s:

```
[servidor-B] === Resincronizacion periodica 5/7 ===
  muestra 1: el servidor de tiempo NO responde (TaskCanceledException: ... Timeout of 4 seconds elapsing.)
  muestra 2: el servidor de tiempo NO responde (HttpRequestException: Name or service not known (time-server:80
80))
  muestra 3: el servidor de tiempo NO responde (HttpRequestException: Connection refused (time-server:8080))
  sin muestras validas: se mantiene el reloj (correccion acumulada -6.081 s). Error real actual: +53.0 ms
[servidor-B] === Resincronizacion periodica 6/7 ===
  antes 10:00:32.910 | despues 10:00:32.837 | ajuste -0.073 s | error real -0.1 ms

[servidor-C] muestra 2: ... RTT=3998.0 ms  -> DESCARTADA (RTT mayor al limite)
[servidor-A] muestra 3: ... RTT=129.9 ms -> ... ajuste +0.042 s | error real +42.1 ms   (primera respuesta tras
reiniciar)
```

¿Cómo influye la latencia de red en la precisión de la sincronización? El cliente no sabe en qué momento del RTT el servidor leyó su reloj; supone que fue justo en la mitad. Por eso el error máximo es **±RTT/2**: a mayor latencia, peor precisión. Con latencia **simétrica** la suposición se cumple y el error sigue siendo casi cero aunque el RTT sea de 400 ms (−0,2 ms). Con latencia **asimétrica** la suposición falla y el error es (ida − vuelta)/2: con 50 ms de ida y 450 ms de vuelta dio exactamente **−201,3 ms**, dentro de la cota de ±252 ms. La variabilidad (*jitter*) también afecta: por eso se toman varias muestras y se usa la de menor RTT. La primera muestra de cada cliente tardó 33–85 ms (conexión TCP nueva, JIT) y quedó descartada frente a la de 1 ms.

¿Qué sucede si el tiempo de transmisión de ida y vuelta es muy elevado? La cota de error ±RTT/2 se vuelve inaceptable (con RTT = 3 s, ±1,5 s) y además la latencia suele ser más irregular, así que la muestra no sirve. El cliente implementado descarta las muestras con RTT mayor a `MAX_RTT_MS` (1000 ms): en la fase “alta” las tres muestras (≈3002 ms) se descartaron y el reloj **no se tocó**, conservando la última corrección válida. Es mejor no ajustar que ajustar con una estimación mala. También pasó en la recuperación del servidor C, cuya muestra de 3998 ms se descartó. Si el RTT alto fuera permanente habría que buscar un servidor más cercano, ampliar el límite aceptando menos precisión, o usar NTP, que filtra y combina muchas muestras.

¿Qué ocurre si el servidor de tiempo deja de responder? Los clientes no pueden sincronizar: las peticiones fallan por *timeout* (4 s) o por conexión rechazada o nombre no resuelto, como muestra el registro. El cliente implementado **sigue funcionando con su última corrección**, pero su reloj **vuelve a derivar** sin control: el servidor B pasó de un error de −0,0 ms a **+53 ms** en los ~10 s que estuvo sin sincronizar (5 ms/s × 10,6 s). Cuando el servidor vuelve, la siguiente resincronización corrige el error acumulado (`ajuste -0.073 s`, error −0,1 ms). Es el punto débil del algoritmo: un único servidor es punto único de falla. Se mitiga con varios servidores de tiempo (consultar a varios y quedarse con la mediana, como hace NTP), reintentos con *backoff* y alertas cuando la última sincronización exitosa es demasiado vieja.

Ejercicio 2 — Algoritmo de Berkeley

Implementación. Cuatro aplicaciones en contenedores: `coordinador` (`Berkeley.Coordinator`) y `nodo-1`, `nodo-2`, `nodo-3` (`Berkeley.Node`, una Web API con `GET /api/clock` y `POST /api/clock/adjust`). En cada ronda el coordinador:

1. sondea a todos los nodos en paralelo (`GET /api/clock`, con timeout de 2 s);
2. calcula la diferencia de cada uno con su propio reloj, compensando la latencia: `hora del nodo + RTT/2 - hora del coordinador`;
3. promedia las diferencias **incluyendo la suya (0)** (`Berkeley.Compute` en `src/Clocks/Berkeley.cs`);
4. calcula el ajuste de cada uno: `promedio - diferencia`;
5. envía a cada nodo **su ajuste** (`POST /api/clock/adjust`) y aplica el propio;
6. repite cada `INTERVAL_SECONDS` (5 s por defecto).

Con `OUTLIER_THRESHOLD_SECONDS > 0` usa el **promedio tolerante a fallas** del algoritmo original: los relojes que difieren de la mediana más que el umbral no entran en el promedio, pero igual reciben su corrección. Un nodo que no responde queda fuera de la ronda. `nodo-2` deriva 2000 ppm (2 ms por segundo) para que se vea la necesidad de resincronizar.

Escenarios 1 a 3: diferencias iniciales de 5, 10 y 20 segundos (`docs/salidas/ej2_berkeley.txt`):

```
Escenario 3 - Relojes iniciales: coordinador 0 s, nodo-1 20 s, nodo-2 -12 s, nodo-3 8 s
===== Ronda 1 =====
participante  hora           diferencia   en promedio  ajuste       hora ajustada
coordinador   10:00:00.828   +0.000 s     si           +4.031 s     10:00:04.859
nodo-1        10:00:20.873   +20.045 s    si           -16.013 s    10:00:04.859
nodo-2        09:59:48.872   -11.956 s    si           +15.987 s    10:00:04.859
nodo-3        10:00:08.864   +8.036 s     si           -4.005 s     10:00:04.859
promedio de diferencias: +4.031 s | dispersion antes del ajuste (max - min): +32.000 s
===== Ronda 2 =====
coordinador   10:00:09.920   +0.000 s     si           -0.028 s     10:00:09.892
nodo-1        10:00:09.876   -0.045 s     si           +0.016 s     10:00:09.892
...
promedio de diferencias: -0.028 s | dispersion antes del ajuste (max - min): +0.045 s
```

Escenario	Desfases iniciales (nodo-1, nodo-2, nodo-3)	Promedio	Dispersión antes	Dispersión en la ronda 2
5 s	+5, −3, +2	+1,024 s	8,008 s	0,038 s
10 s	+10, −6, +4	+2,029 s	16,010 s	0,047 s
20 s	+20, −12, +8	+4,031 s	32,000 s	0,045 s

En todos los casos una ronda alcanza para llevar la dispersión de segundos a **menos de 50 ms**, y la segunda la corrige otra vez. La diferencia residual viene del error de la primera estimación: la primera petición a cada nodo tarda unos 200 ms (conexión nueva, JIT), así que la compensación RTT/2 tiene un error de decenas de ms. El resultado no depende del tamaño de la diferencia inicial: con 20 s queda igual de bien que con 5 s. Notar que **nadie va a la hora del coordinador**: todos van al promedio, que está +1, +2 y +4 s por delante de él. Berkeley logra acuerdo, no exactitud.

Escenarios 4 y 5: un nodo con 5 minutos de diferencia

```
Escenario 4 - promedio simple
coordinador   10:00:00.866   +0.000 s     si           +75.538 s    10:01:16.404
nodo-1        10:00:05.922   +5.056 s     si           +70.482 s    10:01:16.404
nodo-2        09:59:57.918   -2.948 s     si           +78.486 s    10:01:16.404
nodo-3        10:05:00.910   +300.043 s   si           -224.505 s   10:01:16.404
promedio de diferencias: +75.538 s
```

```
Escenario 5 - promedio tolerante a fallas (umbral 30 s)
coordinador   10:00:00.932   +0.000 s     si           +0.696 s     10:00:01.628
nodo-1        10:00:05.973   +5.041 s     si           -4.345 s     10:00:01.628
nodo-2        09:59:57.979   -2.954 s     si           +3.649 s     10:00:01.628
nodo-3        10:05:00.977   +300.045 s   NO (atipico) -299.349 s   10:00:01.628
promedio de diferencias: +0.696 s
```

Con el promedio simple, el nodo desfasado **arrastra a todo el grupo**: la diferencia de 300 s pesa un cuarto en el promedio, así que tres relojes que estaban dentro de ±5 s saltan **75,5 s hacia adelante** para encontrarse con él a mitad de camino. Quedan sincronizados entre sí (dispersión de 56 ms en la ronda 2), pero todos con 75 s de error. Con el promedio tolerante a fallas, nodo-3 se marca como atípico y queda fuera del promedio (+0,696 s, casi el de los tres relojes sanos), pero igual recibe su corrección de −299,3 s y se integra al grupo.

Escenario 6: desconexión de un participante. `nodo-3` se detiene después de la ronda 1 y se reinicia antes de la ronda 4:

```
===== Ronda 2 =====
coordinador   10:00:10.174   +0.000 s     si           -0.030 s     10:00:10.144
nodo-1        10:00:10.113   -0.061 s     si           +0.031 s     10:00:10.144
nodo-2        10:00:10.145   -0.030 s     si           -0.001 s     10:00:10.144
nodo-3        SIN RESPUESTA  -            no           -            -  (timeout)
===== Ronda 4 =====   (nodo-3 vuelve con su reloj original: +4 s)
coordinador   10:00:22.253   +0.000 s     si           +0.509 s     10:00:22.762
nodo-1        10:00:22.252   -0.000 s     si           +0.510 s     10:00:22.762
nodo-2        10:00:22.267   +0.014 s     si           +0.496 s     10:00:22.762
nodo-3        10:00:24.277   +2.024 s     si           -1.515 s     10:00:22.762
```

Mientras nodo-3 no responde, el coordinador espera el timeout (2 s), lo excluye de la ronda y sincroniza al resto sin problemas: el algoritmo **degrada sin detenerse**. Cuando nodo-3 vuelve, se reinicia con su desfase original y vuelve a entrar al promedio. Como el promedio es simple, sus +2 s **corren a todos los nodos sanos +0,5 s**: otro ejemplo de por qué conviene el promedio tolerante a fallas, o hacer que un nodo que se reincorpora primero se sincronice sin votar.

Escenario 7: falla del coordinador. Se detuvo el coordinador después de la ronda 3, con `nodo-2` derivando 20000 ppm (20 ms por segundo), y se leyeron los relojes desde el host:

```
Con coordinador (tras la ronda 3)  nodo-1 10:00:14.756 (+0 ms) | nodo-2 10:00:14.795 (+40 ms)  | nodo-3 (+9 ms)
| dispersion 40 ms
--- se detiene el coordinador ---
Sin coordinador, +5 s              nodo-1 10:00:20.566 (+0 ms) | nodo-2 10:00:20.719 (+154 ms) | nodo-3 (+7 ms)
| dispersion 154 ms
Sin coordinador, +10 s             nodo-1 10:00:25.801 (+0 ms) | nodo-2 10:00:26.073 (+272 ms) | nodo-3 (+19 m
s) | dispersion 272 ms
Sin coordinador, +20 s             nodo-1 10:00:36.100 (+0 ms) | nodo-2 10:00:36.578 (+478 ms) | nodo-3 (+19 m
s) | dispersion 478 ms
```

Los nodos siguen funcionando, pero nadie los corrige: la dispersión crece al ritmo de la deriva, unos 20 ms por segundo, de 40 a 478 ms en 20 s.

¿Qué diferencias existen entre Cristian y Berkeley? Cristian es **cliente–servidor**: cada cliente le *pide* la hora a un servidor que se supone correcto (con acceso a UTC) y la copia, así que es **sincronización externa** y solo se ajustan los clientes. Berkeley es **coordinado**: el coordinador *sondea* a todos, nadie tiene la hora correcta, se calcula el **promedio** y **todos se ajustan**, el coordinador incluido. Es **sincronización interna**: el grupo puede quedar de acuerdo pero lejos de UTC, como en los escenarios 1 a 3, donde todos quedaron entre 1 y 4 s por delante del coordinador. Cristian manda la hora; Berkeley manda ajustes relativos. Berkeley puede filtrar relojes defectuosos con el promedio tolerante a fallas; Cristian copia lo que diga el servidor, esté bien o mal. Ver también la tabla de la pregunta teórica 7.

¿Qué sucede cuando un nodo presenta un reloj considerablemente desfasado? Con el **promedio simple**, ese reloj **contamina el resultado**: su diferencia entra con el mismo peso que las demás y mueve a todo el grupo hacia él. Con 5 minutos de desfase en uno de cuatro relojes, todos saltaron +75,5 s, una corrección enorme y equivocada para tres relojes que estaban bien. Además, un salto así hacia adelante (o hacia atrás para el nodo desfasado) puede romper timeouts, vencimientos y el orden de los logs. La solución es el **promedio tolerante a fallas** del algoritmo original: considerar solo los relojes que no difieren entre sí (o de la mediana) más que un umbral. En el escenario 5 el nodo quedó fuera del promedio (+0,696 s) y aun así se corrigió. Otras opciones son usar la **mediana** directamente, limitar el tamaño máximo de un ajuste, o corregir gradualmente (*slewing*) en vez de saltar.

¿Qué ocurre si el coordinador deja de funcionar? Se detiene la sincronización: no hay quién sondee ni calcule ajustes. Los nodos **siguen funcionando** con su último ajuste, pero sus relojes **vuelven a derivar** y la dispersión crece sin control (escenario 7: de 40 a 478 ms en 20 s con un nodo de 20 ms/s). Lo que dependa de que los relojes coincidan (orden de logs, vencimientos, *leases*) se degrada de a poco. El coordinador es un punto único de falla.

¿Qué modificaciones permitirían tolerar la falla del coordinador?

- **Elección de un nuevo coordinador**: los nodos detectan la falla (no llega la ronda esperada dentro de un plazo, o fallan *heartbeats*) y ejecutan un algoritmo de elección, como el del matón (*bully*), el de anillo o Raft. Cualquier nodo puede ser coordinador porque el algoritmo no necesita una fuente de hora especial. El demonio `timed` de Berkeley UNIX ya elegía un nuevo maestro cuando el anterior fallaba.
- **Réplicas en espera**: un coordinador secundario que monitorea al primario y toma su lugar.
- **Sincronización descentralizada**: que cada nodo sondee a los demás y aplique él mismo el promedio tolerante a fallas, sin coordinador (como los algoritmos de promedio distribuido), a costa de más mensajes.
- **Respaldo externo**: que cada nodo, además, use NTP contra varios servidores para no derivar sin límite mientras no hay coordinador.
- **Estado persistente o idempotente**: que los ajustes sean relativos (ya lo son) y que un coordinador nuevo pueda empezar una ronda desde cero sin conocer las anteriores.

Ejercicio 3 — Relojes lógicos de Lamport

Implementación. Tres Web API (`Lamport.Orders`, `Lamport.Inventory` y `Lamport.Payments`) que se comunican por HTTP REST. Cada una tiene un `EventProcess` con su reloj físico, su contador de Lamport y, como herramienta de análisis, un reloj vectorial. Las tres reglas están en `src/Clocks/LogicalClocks.cs`:

```
public long Tick()                    { lock (_lock) return ++_value; }                          // evento inter
no / envio
public long Receive(long received)    { lock (_lock) { _value = Math.Max(_value, received) + 1; return _value;
} }   // recepcion
```

Cada mensaje (petición y respuesta) lleva el reloj en los headers `X-Lamport-Clock` y `X-Vector-Clock`. Los relojes físicos están desfasados a propósito: Orders +0 s, Inventory **−3 s**, Payments **+5 s**. Payments arranca después de procesar 4 pagos de otros pedidos, así que su contador empieza en 4. Se enviaron dos pedidos: `P001` (con stock) y `P003` (sin stock).

Resultados (`docs/salidas/ej3_lamport.txt`). Tabla pedida para el primer pedido:

Paso	Proceso	Evento	Reloj físico	Reloj lógico
1	OrdersService	Pedido recibido	10:00:00.091	1
2	OrdersService	Mensaje enviado a InventoryService	10:00:00.097	2
3	InventoryService	Mensaje recibido (recibido L=2 > local 0: salta a 3)	**09:59:57.228**	3
4	InventoryService	Inventario verificado: P001 stock 10, pedido 2 → reservado	09:59:57.229	4
5	InventoryService	Respuesta enviada	09:59:57.229	5
—	OrdersService	Respuesta recibida (recibido L=5 > local 2: salta a 6)	10:00:00.245	6
6	OrdersService	Solicitud de pago enviada	10:00:00.252	7
7	PaymentsService	Solicitud de pago recibida (recibido L=7 > local 4: salta a 8)	10:00:05.356	8
7	PaymentsService	Pago procesado: $150.00 aprobado	10:00:05.408	9
8	PaymentsService	Confirmación enviada	10:00:05.408	10
—	OrdersService	Confirmación de pago recibida (recibido L=10 > local 7: salta a 11)	**10:00:00.418**	11
9	OrdersService	Pedido ORD-001 confirmado	10:00:00.419	12

El segundo pedido continúa la numeración (Orders 13–19, Inventory 15–17) y termina en “Pedido ORD-002 rechazado: sin stock” con L=19.

`GET /api/timeline` junta los registros de los tres servicios y compara los dos órdenes:

```
=== Orden por reloj fisico (hora local de cada servicio) ===
InventoryService  09:59:57.228   3   Mensaje recibido de OrdersService (pedido ORD-001)
InventoryService  09:59:57.229   4   Inventario verificado ...
...
OrdersService     10:00:00.091   1   Pedido recibido ORD-001       <- la causa aparece DESPUES del efecto
OrdersService     10:00:00.097   2   Mensaje enviado a InventoryService

=== Inconsistencias del reloj fisico: 93 pares donde a -> b pero hora(b) < hora(a) ===
  Orders L=2 "Mensaje enviado a InventoryService"     ->  Inventory L=3 "Mensaje recibido"   (hora(b) - hora(a)
= -2.861 s)
  Payments L=10 "Confirmacion enviada a OrdersService" ->  Orders L=11 "Confirmacion recibida" (hora(b) - hora
(a) = -4.990 s)
  Orders L=14 "Mensaje enviado a InventoryService"    ->  Inventory L=15 "Mensaje recibido"  (hora(b) - hora(a)
= -2.997 s)

=== Eventos concurrentes con L(a) < L(b): 24 pares (Lamport no los distingue) ===
  Orders L=1 [I=0,O=1,P=0] "Pedido recibido ORD-001"  ||  Payments L=2 [I=0,O=0,P=2] "Pago de otro pedido proce
sado (#2)"
  Orders L=2 [I=0,O=2,P=0] "Mensaje enviado a ..."    ||  Payments L=3 [I=0,O=0,P=3] "Pago de otro pedido proce
sado (#3)"
```

Ordenado por Lamport, cada envío aparece antes de su recepción (2 → 3, 5 → 6, 7 → 8, 10 → 11, 14 → 15, 17 → 18): ningún par viola la causalidad.

¿Por qué los valores de Lamport no representan la hora real? Porque son **contadores de eventos**, no mediciones de tiempo: solo aumentan cuando ocurre algo (un evento interno, un envío o una recepción), no con el paso del tiempo. No tienen unidad ni relación con UTC: entre L=9 y L=10 de Payments pasaron 0 ms, entre L=1 y L=2 de Orders 6 ms, y entre los dos pedidos casi 50 ms, sin que eso se refleje en los números. Además, al recibir un mensaje el contador salta para respetar la causalidad: Inventory pasó de 0 a 3 sin que ocurrieran tres eventos. Lo único que garantizan es el orden: si $a \rightarrow b$ entonces $L(a) < L(b)$.

¿Qué ocurre cuando un proceso recibe un mensaje con un valor lógico superior al suyo? Adelanta su contador para quedar por delante del emisor: $C = max(C, t) + 1 = t + 1$. Así la recepción queda numerada después del envío, aunque el receptor haya tenido menos actividad. En la práctica: Inventory estaba en 0, recibió L=2 y pasó a **3**; Orders estaba en 2, recibió la respuesta con L=5 y pasó a **6**; Payments estaba en 4, recibió L=7 y pasó a **8**. Si el valor recibido es menor o igual al propio, solo se suma 1. Por ejemplo, si Payments hubiera procesado 10 pagos previos (L=10) y recibido L=7, pasaría a 11. Así el reloj nunca retrocede.

¿Cómo permite Lamport mantener el orden causal de los eventos? Garantiza la **condición de reloj**: si $a \rightarrow b$ entonces $L(a) < L(b)$. Dentro de un proceso, cada evento incrementa el contador, así que los eventos posteriores tienen valores mayores. Entre procesos, el mensaje lleva el valor del envío y el receptor queda por encima con max + 1, así que la recepción es mayor que el envío. Por transitividad, cualquier cadena causal tiene valores crecientes. Por eso ordenar por $(L, proceso)$ produce un orden total **consistente con la causalidad**, que todos los procesos pueden calcular igual, sin importar sus relojes físicos. En el registro, ese orden puso los 9 pasos del pedido en la secuencia correcta, mientras que el orden físico tenía 93 pares invertidos.

¿Es posible determinar si dos eventos son concurrentes utilizando únicamente sus marcas de tiempo de Lamport? No. La implicación vale en un solo sentido: $a \rightarrow b \Rightarrow L(a) < L(b)$, pero **L(a) < L(b) no implica a → b**. Dos eventos concurrentes pueden tener cualquier par de valores. En la salida, “Pedido recibido ORD-001” (Orders, L=1) y “Pago de otro pedido #2” (Payments, L=2) cumplen 1 < 2 y sin embargo son **concurrentes**: no hay ningún mensaje que los conecte. Hubo 24 pares así. Lo único que se puede afirmar es que si $L(a) \geq L(b)$, entonces a no ocurrió antes que b. Para detectar concurrencia se usan **relojes vectoriales**: cada proceso guarda un contador por proceso y $a \rightarrow b$ si y solo si $V(a) \leq V(b)$ componente a componente (y son distintos). Los vectores `[I=0,O=1,P=0]` y `[I=0,O=0,P=2]` no son comparables, por lo tanto los eventos son concurrentes. Por eso el proyecto los incluye como herramienta de análisis.

Ejercicio 4 — Comparación de algoritmos

Característica	Cristian	Berkeley	Lamport
Tipo de reloj	Físico	Físico	Lógico (contador)
Objetivo principal	Acercar la hora de cada cliente a la de un servidor de referencia (sincronización externa, UTC)	Que todos los relojes del grupo coincidan entre sí (sincronización interna)	Ordenar los eventos respetando la causalidad (*happens-before*)
Requiere servidor central	Sí: el servidor de tiempo	Sí: el coordinador (elegible, no tiene la hora "correcta")	No: cada proceso mantiene su contador
Utiliza mensajes	Sí: petición y respuesta (2 por sincronización y cliente)	Sí: sondeo y ajuste (≈3 por nodo y ronda)	Sí, pero no mensajes propios: el contador viaja dentro de los mensajes de la aplicación (un header)
Sincroniza hora física	Sí, con precisión ±RTT/2 (0,1 ms en Docker, −201 ms con latencia asimétrica)	Sí, entre los participantes; el promedio puede alejarse de UTC	No: no tiene relación con la hora real
Establece orden causal	No: tras sincronizar puede seguir habiendo eventos invertidos más cercanos que la precisión	No, por la misma razón	Sí: si a → b entonces L(a) < L(b); no detecta concurrencia (para eso, relojes vectoriales)
Comportamiento ante fallas	Si cae el servidor, nadie sincroniza y los relojes vuelven a derivar (B: +53 ms en ~10 s); se descartan las muestras con RTT alto; un servidor con hora errónea contagia a todos	Un nodo caído se excluye de la ronda; un reloj muy desviado arrastra el promedio salvo con el promedio tolerante a fallas; si cae el coordinador nadie sincroniza hasta elegir otro	No depende de un servidor; si un proceso cae, el resto sigue. Exige que los mensajes lleven el contador; un proceso que reinicia en 0 debe persistir su contador o recibirlo de los demás

1. ¿Qué algoritmo utilizaría para sincronizar los relojes de varios servidores? Si hay acceso a una referencia confiable de UTC, el enfoque de **Cristian**, en la práctica **NTP** (que es Cristian refinado: varias muestras, varios servidores, filtrado y corrección gradual) o **PTP** (IEEE 1588) si se necesita precisión de microsegundos. Es lo que se usa en cualquier centro de datos. Si los servidores están aislados y no tienen referencia externa, o lo único que importa es que coincidan entre ellos, **Berkeley** con promedio tolerante a fallas y elección de coordinador. Lo que no haría es depender de un único servidor de tiempo sin redundancia.

2. ¿Qué algoritmo utilizaría para establecer el orden causal de operaciones entre microservicios? Relojes lógicos: **Lamport** si alcanza con un orden total consistente con la causalidad (por ejemplo, para ordenar eventos en un log, versionar operaciones o desempatar en exclusión mutua distribuida), o **relojes vectoriales** si además hay que detectar operaciones concurrentes (conflictos en réplicas, como hacen Dynamo o Riak). No usaría relojes físicos para esto: el ejercicio 3 mostró 93 pares de eventos invertidos con desfases de apenas 3 y 5 s. Además, el contador viaja en un header, sin servidores ni mensajes extra, lo que encaja naturalmente en HTTP o en colas de mensajes. Lo mismo hacen en la práctica los identificadores de trazas distribuidas o los *hybrid logical clocks* de bases como CockroachDB.

3. ¿Qué diferencias existen entre sincronización física y sincronización lógica? La **física** ajusta relojes que miden tiempo real para que coincidan con UTC o entre sí, siempre con un error residual (±RTT/2, deriva entre sincronizaciones), y sirve para todo lo que necesita la hora o las duraciones: auditoría, vencimientos, timeouts, métricas. La **lógica** no mide tiempo: numera eventos para capturar la relación *happens-before*, es exacta respecto de la causalidad, no necesita servidores ni mensajes de sincronización y no sufre deriva, pero

no dice qué hora es ni cuánto tardó algo. La física responde “¿cuándo?”, la lógica responde “¿qué pasó antes?”. Ver también la tabla de la pregunta 11.

4. ¿Es posible combinar diferentes algoritmos en una misma arquitectura distribuida? Justifique. Sí, y es lo habitual, porque resuelven problemas distintos y complementarios:

- **NTP (Cristian) + Berkeley**: los servidores se sincronizan con UTC vía NTP y, dentro de un clúster aislado, un *time daemon* tipo Berkeley mantiene la coherencia interna si se pierde la referencia externa.
- **Relojes físicos + lógicos**: la plataforma de comercio de este caso usaría NTP para que las fechas de pedidos, facturas y logs sean razonables para las personas y para los vencimientos, y Lamport o vectoriales en los mensajes entre `OrdersService`, `InventoryService` y `PaymentsService` para que el orden de las operaciones sea correcto aunque los relojes difieran.
- **Relojes híbridos (HLC)**: combinan las dos ideas en una sola marca, *(hora física, contador lógico)*. Se mantienen cerca de la hora real pero garantizan la condición de reloj de Lamport. Los usan bases de datos distribuidas como CockroachDB y MongoDB. Google Spanner va más allá con TrueTime: relojes atómicos y GPS que acotan la incertidumbre y esperan a que pase.

La condición es usar cada mecanismo para lo que sirve: nunca ordenar causalmente por hora física, y nunca sacar duraciones u horas de un reloj lógico.

Tests

`dotnet test tests/Clocks.Tests` ejecuta 10 tests de la lógica pura (sin red): estimación de Cristian y selección de la mejor muestra, promedio de Berkeley con nodo atípico, promedio tolerante a fallas y nodo desconectado, reglas de Lamport (salto al recibir un valor mayor y +1 al recibir uno menor) y detección de concurrencia con relojes vectoriales. Resultado en `docs/salidas/tests.txt`.

Bibliografía

1. Tanenbaum, A. S. y Van Steen, M. *Distributed Systems*, cap. 6 “Coordination”: sincronización de relojes, relojes lógicos, relojes vectoriales.
2. Coulouris, G., Dollimore, J., Kindberg, T. y Blair, G. *Distributed Systems: Concepts and Design*, cap. 14 “Time and Global States”.
3. Cristian, F. (1989). *Probabilistic clock synchronization*. Distributed Computing 3(3).
4. Gusella, R. y Zatti, S. (1989). *The accuracy of the clock synchronization achieved by TEMPO in Berkeley UNIX 4.3BSD*. IEEE TSE 15(7).
5. Lamport, L. (1978). *Time, clocks, and the ordering of events in a distributed system*. Communications of the ACM 21(7).
6. Mills, D. et al. RFC 5905 — *Network Time Protocol Version 4*.
7. Microsoft Learn — ASP.NET Core Minimal APIs, `HttpClient`, Docker para .NET.

Anexo A — Salidas completas de los scripts

Registros de ejecución generados con los scripts de `scripts/`. Son las salidas reales a partir de las cuales se respondieron los ejercicios.

Tests unitarios (dotnet test)

```
Correctas Clocks.Tests.SimulatedClockTests.Drift_accumulates_and_adjust_changes_only_the_application_clock [62
ms]
Correctas Clocks.Tests.CristianTests.Best_sample_is_the_lowest_rtt_and_slow_samples_are_discarded [62 ms]
Correctas Clocks.Tests.BerkeleyTests.Fault_tolerant_average_ignores_the_outlier_but_still_corrects_it [64 ms]
Correctas Clocks.Tests.SimulatedClockTests.Shows_the_case_study_times_at_the_scenario_start [< 1 ms]
Correctas Clocks.Tests.BerkeleyTests.Disconnected_participants_are_left_out_and_get_no_adjustment [< 1 ms]
Correctas Clocks.Tests.CristianTests.Estimates_server_time_plus_half_the_round_trip [< 1 ms]
Correctas Clocks.Tests.BerkeleyTests.A_five_minute_outlier_drags_the_plain_average [1 ms]
Correctas Clocks.Tests.LogicalClockTests.Vector_clocks_detect_concurrency_that_lamport_cannot [68 ms]
Correctas Clocks.Tests.LogicalClockTests.Lamport_ticks_on_local_events_and_takes_the_max_on_receive [2 ms]
Correctas Clocks.Tests.BerkeleyTests.Everyone_moves_to_the_average_including_the_coordinator [10 ms]
```

Ejercicio 1 — Cristian

```
Escenario: hora simulada 10:00:00 = 2026-11-11T20:31:53.071Z (UTC real)
Servidor de tiempo detenido a las 20:32:10 y reiniciado a las 20:32:19 (UTC real), ~9 s sin servicio

===== time-server =====

[time-server] servidor de tiempo iniciado, hora local 10:00:01.751
[time-server] hora 10:00:02.055 -> servidor-A
[time-server] hora 10:00:02.127 -> servidor-A
[time-server] hora 10:00:02.128 -> servidor-A
[time-server] hora 10:00:02.154 -> servidor-C
[time-server] hora 10:00:02.176 -> servidor-B
[time-server] hora 10:00:02.190 -> servidor-C
[time-server] hora 10:00:02.192 -> servidor-C
[time-server] hora 10:00:02.208 -> servidor-B
[time-server] hora 10:00:02.210 -> servidor-B
[time-server] hora 10:00:02.337 -> servidor-A (latencia simulada: ida 200 ms, vuelta 200 ms)
[time-server] hora 10:00:02.739 -> servidor-A (latencia simulada: ida 200 ms, vuelta 200 ms)
[time-server] hora 10:00:03.141 -> servidor-A (latencia simulada: ida 200 ms, vuelta 200 ms)
[time-server] hora 10:00:03.393 -> servidor-A (latencia simulada: ida 50 ms, vuelta 450 ms)
[time-server] hora 10:00:03.898 -> servidor-A (latencia simulada: ida 50 ms, vuelta 450 ms)
[time-server] hora 10:00:04.402 -> servidor-A (latencia simulada: ida 50 ms, vuelta 450 ms)
[time-server] hora 10:00:06.197 -> servidor-C
[time-server] hora 10:00:06.198 -> servidor-C
[time-server] hora 10:00:06.198 -> servidor-C
[time-server] hora 10:00:06.215 -> servidor-B
[time-server] hora 10:00:06.216 -> servidor-B
[time-server] hora 10:00:06.217 -> servidor-B
[time-server] hora 10:00:06.356 -> servidor-A (latencia simulada: ida 1500 ms, vuelta 1500 ms)
[time-server] hora 10:00:09.358 -> servidor-A (latencia simulada: ida 1500 ms, vuelta 1500 ms)
[time-server] hora 10:00:10.199 -> servidor-C
[time-server] hora 10:00:10.200 -> servidor-C
[time-server] hora 10:00:10.201 -> servidor-C
[time-server] hora 10:00:10.220 -> servidor-B
[time-server] hora 10:00:10.221 -> servidor-B
[time-server] hora 10:00:10.222 -> servidor-B
[time-server] hora 10:00:12.360 -> servidor-A (latencia simulada: ida 1500 ms, vuelta 1500 ms)
[time-server] hora 10:00:14.203 -> servidor-C
[time-server] hora 10:00:14.204 -> servidor-C
[time-server] hora 10:00:14.204 -> servidor-C
[time-server] hora 10:00:14.225 -> servidor-B
[time-server] hora 10:00:14.226 -> servidor-B
[time-server] hora 10:00:14.227 -> servidor-B
[time-server] hora 10:00:17.863 -> servidor-A
```

```
[time-server] hora 10:00:17.864 -> servidor-A
[time-server] hora 10:00:17.865 -> servidor-A
[time-server] hora 10:00:18.205 -> servidor-C
[time-server] hora 10:00:18.207 -> servidor-C
[time-server] hora 10:00:18.207 -> servidor-C
[time-server] hora 10:00:18.228 -> servidor-B
[time-server] hora 10:00:18.229 -> servidor-B
[time-server] hora 10:00:18.229 -> servidor-B
[time-server] servidor de tiempo iniciado, hora local 10:00:29.112
[time-server] hora 10:00:29.979 -> servidor-A
[time-server] hora 10:00:30.208 -> servidor-C
[time-server] hora 10:00:30.211 -> servidor-C
[time-server] hora 10:00:32.835 -> servidor-B
[time-server] hora 10:00:32.836 -> servidor-B
[time-server] hora 10:00:32.837 -> servidor-B
[time-server] hora 10:00:34.002 -> servidor-A
[time-server] hora 10:00:34.003 -> servidor-A
[time-server] hora 10:00:34.004 -> servidor-A
[time-server] hora 10:00:34.212 -> servidor-C
[time-server] hora 10:00:34.213 -> servidor-C
[time-server] hora 10:00:34.214 -> servidor-C
[time-server] hora 10:00:36.841 -> servidor-B
[time-server] hora 10:00:36.842 -> servidor-B
[time-server] hora 10:00:36.843 -> servidor-B
[time-server] hora 10:00:38.003 -> servidor-A
[time-server] hora 10:00:38.004 -> servidor-A
[time-server] hora 10:00:38.004 -> servidor-A
[time-server] hora 10:00:38.214 -> servidor-C
[time-server] hora 10:00:38.215 -> servidor-C
[time-server] hora 10:00:38.216 -> servidor-C
[time-server] hora 10:00:42.007 -> servidor-A
[time-server] hora 10:00:42.008 -> servidor-A
[time-server] hora 10:00:42.009 -> servidor-A
[time-server] hora 10:00:46.012 -> servidor-A
[time-server] hora 10:00:46.014 -> servidor-A
[time-server] hora 10:00:46.014 -> servidor-A
[time-server] hora 10:00:50.020 -> servidor-A
[time-server] hora 10:00:50.020 -> servidor-A
[time-server] hora 10:00:50.021 -> servidor-A

===== client-a =====

[servidor-A 09:59:59.815] iniciado. Hora local 09:59:59.808 | servidor http://time-server:8080 | deriva 0 ppm
[servidor-A 09:59:59.818] diferencia real con el servidor antes de sincronizar: -2.000 s
[servidor-A 10:00:00.036] === Fase 'normal': red Docker sin latencia agregada ===
[servidor-A 10:00:00.126]   muestra 1: T0=10:00:00.040 T1=10:00:00.125 servidor=10:00:02.055 RTT=85.3 ms
[servidor-A 10:00:00.128]   muestra 2: T0=10:00:00.126 T1=10:00:00.128 servidor=10:00:02.127 RTT=1.8 ms
[servidor-A 10:00:00.129]   muestra 3: T0=10:00:00.128 T1=10:00:00.129 servidor=10:00:02.128 RTT=1.0 ms
[servidor-A 10:00:02.135]   mejor muestra RTT=1.0 ms -> hora estimada = servidor + RTT/2 (precision +/-0.5 ms)
[servidor-A 10:00:02.135]   antes 10:00:00.135 | despues 10:00:02.135 | ajuste +2.000 s | error real -0.1 ms
[servidor-A 10:00:02.135] === Fase 'simetrica': latencia simetrica: 200 ms de ida y 200 ms de vuelta ===
[servidor-A 10:00:02.539]   muestra 1: T0=10:00:02.135 T1=10:00:02.539 servidor=10:00:02.337 RTT=403.2 ms
[servidor-A 10:00:02.940]   muestra 2: T0=10:00:02.539 T1=10:00:02.940 servidor=10:00:02.739 RTT=401.7 ms
[servidor-A 10:00:03.343]   muestra 3: T0=10:00:02.940 T1=10:00:03.342 servidor=10:00:03.141 RTT=402.1 ms
[servidor-A 10:00:03.342]   mejor muestra RTT=401.7 ms -> hora estimada = servidor + RTT/2 (precision +/-200.9
ms)
[servidor-A 10:00:03.342]   antes 10:00:03.343 | despues 10:00:03.342 | ajuste -0.000 s | error real -0.2 ms
[servidor-A 10:00:03.342] === Fase 'asimetrica': latencia asimetrica: 50 ms de ida y 450 ms de vuelta ===
[servidor-A 10:00:03.847]   muestra 1: T0=10:00:03.342 T1=10:00:03.847 servidor=10:00:03.393 RTT=504.2 ms
[servidor-A 10:00:04.351]   muestra 2: T0=10:00:03.847 T1=10:00:04.351 servidor=10:00:03.898 RTT=504.2 ms
[servidor-A 10:00:04.855]   muestra 3: T0=10:00:04.351 T1=10:00:04.855 servidor=10:00:04.402 RTT=504.3 ms
[servidor-A 10:00:04.654]   mejor muestra RTT=504.2 ms -> hora estimada = servidor + RTT/2 (precision +/-252.1
ms)
[servidor-A 10:00:04.654]   antes 10:00:04.855 | despues 10:00:04.654 | ajuste -0.201 s | error real -201.3 ms
[servidor-A 10:00:04.654] === Fase 'alta': RTT muy alto: 1500 ms de ida y 1500 ms de vuelta (limite 1000 ms) ==
=
[servidor-A 10:00:07.656]   muestra 1: T0=10:00:04.654 T1=10:00:07.656 servidor=10:00:06.356 RTT=3001.9 ms  ->
DESCARTADA (RTT mayor al limite)
[servidor-A 10:00:10.658]   muestra 2: T0=10:00:07.656 T1=10:00:10.658 servidor=10:00:09.358 RTT=3002.0 ms  ->
DESCARTADA (RTT mayor al limite)
[servidor-A 10:00:13.660]   muestra 3: T0=10:00:10.658 T1=10:00:13.660 servidor=10:00:12.360 RTT=3001.8 ms  ->
DESCARTADA (RTT mayor al limite)
[servidor-A 10:00:13.660]   sin muestras validas: se mantiene el reloj (correccion acumulada +1.799 s). Error r
```

```
eal actual: -201.3 ms
[servidor-A 10:00:17.661] === Resincronizacion periodica 1/7 ===
[servidor-A 10:00:17.662]   muestra 1: T0=10:00:17.661 T1=10:00:17.662 servidor=10:00:17.863 RTT=1.3 ms
[servidor-A 10:00:17.663]   muestra 2: T0=10:00:17.662 T1=10:00:17.663 servidor=10:00:17.864 RTT=0.6 ms
[servidor-A 10:00:17.664]   muestra 3: T0=10:00:17.663 T1=10:00:17.664 servidor=10:00:17.865 RTT=0.9 ms
[servidor-A 10:00:17.865]   mejor muestra RTT=0.6 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-A 10:00:17.865]   antes 10:00:17.664 | despues 10:00:17.865 | ajuste +0.201 s | error real -0.0 ms
[servidor-A 10:00:21.867] === Resincronizacion periodica 2/7 ===
[servidor-A 10:00:25.871]   muestra 1: el servidor de tiempo NO responde (TaskCanceledException: The request wa
s canceled due to the configured HttpClient.Timeout of 4 seconds elapsing.)
[servidor-A 10:00:29.872]   muestra 2: el servidor de tiempo NO responde (TaskCanceledException: The request wa
s canceled due to the configured HttpClient.Timeout of 4 seconds elapsing.)
[servidor-A 10:00:30.002]   muestra 3: T0=10:00:29.872 T1=10:00:30.002 servidor=10:00:29.979 RTT=129.9 ms
[servidor-A 10:00:30.044]   mejor muestra RTT=129.9 ms -> hora estimada = servidor + RTT/2 (precision +/-65.0 m
s)
[servidor-A 10:00:30.044]   antes 10:00:30.002 | despues 10:00:30.044 | ajuste +0.042 s | error real +42.1 ms
[servidor-A 10:00:34.044] === Resincronizacion periodica 3/7 ===
[servidor-A 10:00:34.045]   muestra 1: T0=10:00:34.044 T1=10:00:34.045 servidor=10:00:34.002 RTT=1.1 ms
[servidor-A 10:00:34.046]   muestra 2: T0=10:00:34.045 T1=10:00:34.046 servidor=10:00:34.003 RTT=0.6 ms
[servidor-A 10:00:34.046]   muestra 3: T0=10:00:34.046 T1=10:00:34.046 servidor=10:00:34.004 RTT=0.6 ms
[servidor-A 10:00:34.004]   mejor muestra RTT=0.6 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-A 10:00:34.004]   antes 10:00:34.046 | despues 10:00:34.004 | ajuste -0.042 s | error real -0.1 ms
[servidor-A 10:00:38.002] === Resincronizacion periodica 4/7 ===
[servidor-A 10:00:38.003]   muestra 1: T0=10:00:38.002 T1=10:00:38.003 servidor=10:00:38.003 RTT=1.2 ms
[servidor-A 10:00:38.004]   muestra 2: T0=10:00:38.003 T1=10:00:38.004 servidor=10:00:38.004 RTT=0.7 ms
[servidor-A 10:00:38.005]   muestra 3: T0=10:00:38.004 T1=10:00:38.005 servidor=10:00:38.004 RTT=1.0 ms
[servidor-A 10:00:38.005]   mejor muestra RTT=0.7 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-A 10:00:38.005]   antes 10:00:38.005 | despues 10:00:38.005 | ajuste +0.000 s | error real -0.0 ms
[servidor-A 10:00:42.007] === Resincronizacion periodica 5/7 ===
[servidor-A 10:00:42.008]   muestra 1: T0=10:00:42.007 T1=10:00:42.008 servidor=10:00:42.007 RTT=1.2 ms
[servidor-A 10:00:42.009]   muestra 2: T0=10:00:42.008 T1=10:00:42.009 servidor=10:00:42.008 RTT=0.7 ms
[servidor-A 10:00:42.009]   muestra 3: T0=10:00:42.009 T1=10:00:42.009 servidor=10:00:42.009 RTT=0.6 ms
[servidor-A 10:00:42.009]   mejor muestra RTT=0.6 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-A 10:00:42.009]   antes 10:00:42.009 | despues 10:00:42.009 | ajuste -0.000 s | error real -0.1 ms
[servidor-A 10:00:46.011] === Resincronizacion periodica 6/7 ===
[servidor-A 10:00:46.013]   muestra 1: T0=10:00:46.012 T1=10:00:46.013 servidor=10:00:46.012 RTT=1.4 ms
[servidor-A 10:00:46.014]   muestra 2: T0=10:00:46.013 T1=10:00:46.014 servidor=10:00:46.014 RTT=1.0 ms
[servidor-A 10:00:46.015]   muestra 3: T0=10:00:46.014 T1=10:00:46.015 servidor=10:00:46.014 RTT=0.8 ms
[servidor-A 10:00:46.015]   mejor muestra RTT=0.8 ms -> hora estimada = servidor + RTT/2 (precision +/-0.4 ms)
[servidor-A 10:00:46.015]   antes 10:00:46.015 | despues 10:00:46.015 | ajuste -0.000 s | error real -0.1 ms
[servidor-A 10:00:50.019] === Resincronizacion periodica 7/7 ===
[servidor-A 10:00:50.020]   muestra 1: T0=10:00:50.019 T1=10:00:50.020 servidor=10:00:50.020 RTT=1.1 ms
[servidor-A 10:00:50.021]   muestra 2: T0=10:00:50.020 T1=10:00:50.021 servidor=10:00:50.020 RTT=0.7 ms
[servidor-A 10:00:50.021]   muestra 3: T0=10:00:50.021 T1=10:00:50.021 servidor=10:00:50.021 RTT=0.6 ms
[servidor-A 10:00:50.022]   mejor muestra RTT=0.6 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-A 10:00:50.022]   antes 10:00:50.021 | despues 10:00:50.022 | ajuste +0.000 s | error real -0.1 ms
[servidor-A 10:00:50.022] fin

===== client-b =====

[servidor-B 10:00:08.083] iniciado. Hora local 10:00:08.070 | servidor http://time-server:8080 | deriva 5000 pp
m
[servidor-B 10:00:08.088] diferencia real con el servidor antes de sincronizar: +6.000 s
[servidor-B 10:00:08.173] === Fase 'normal': red Docker sin latencia agregada ===
[servidor-B 10:00:08.208]   muestra 1: T0=10:00:08.175 T1=10:00:08.208 servidor=10:00:02.176 RTT=32.9 ms
[servidor-B 10:00:08.210]   muestra 2: T0=10:00:08.208 T1=10:00:08.210 servidor=10:00:02.208 RTT=2.2 ms
[servidor-B 10:00:08.212]   muestra 3: T0=10:00:08.210 T1=10:00:08.212 servidor=10:00:02.210 RTT=1.2 ms
[servidor-B 10:00:02.214]   mejor muestra RTT=1.2 ms -> hora estimada = servidor + RTT/2 (precision +/-0.6 ms)
[servidor-B 10:00:02.214]   antes 10:00:08.215 | despues 10:00:02.214 | ajuste -6.001 s | error real -0.1 ms
[servidor-B 10:00:06.235] === Resincronizacion periodica 1/7 ===
[servidor-B 10:00:06.236]   muestra 1: T0=10:00:06.235 T1=10:00:06.236 servidor=10:00:06.215 RTT=1.3 ms
[servidor-B 10:00:06.237]   muestra 2: T0=10:00:06.236 T1=10:00:06.237 servidor=10:00:06.216 RTT=0.8 ms
[servidor-B 10:00:06.238]   muestra 3: T0=10:00:06.237 T1=10:00:06.238 servidor=10:00:06.217 RTT=0.9 ms
[servidor-B 10:00:06.218]   mejor muestra RTT=0.8 ms -> hora estimada = servidor + RTT/2 (precision +/-0.4 ms)
[servidor-B 10:00:06.218]   antes 10:00:06.238 | despues 10:00:06.218 | ajuste -0.020 s | error real -0.1 ms
[servidor-B 10:00:10.239] === Resincronizacion periodica 2/7 ===
[servidor-B 10:00:10.240]   muestra 1: T0=10:00:10.239 T1=10:00:10.240 servidor=10:00:10.220 RTT=1.1 ms
[servidor-B 10:00:10.241]   muestra 2: T0=10:00:10.240 T1=10:00:10.241 servidor=10:00:10.221 RTT=0.8 ms
[servidor-B 10:00:10.242]   muestra 3: T0=10:00:10.241 T1=10:00:10.242 servidor=10:00:10.222 RTT=0.9 ms
[servidor-B 10:00:10.222]   mejor muestra RTT=0.8 ms -> hora estimada = servidor + RTT/2 (precision +/-0.4 ms)
[servidor-B 10:00:10.222]   antes 10:00:10.242 | despues 10:00:10.222 | ajuste -0.020 s | error real -0.1 ms
[servidor-B 10:00:14.244] === Resincronizacion periodica 3/7 ===
[servidor-B 10:00:14.245]   muestra 1: T0=10:00:14.244 T1=10:00:14.245 servidor=10:00:14.225 RTT=1.2 ms
```

```
[servidor-B 10:00:14.246]   muestra 2: T0=10:00:14.245 T1=10:00:14.246 servidor=10:00:14.226 RTT=0.8 ms
[servidor-B 10:00:14.247]   muestra 3: T0=10:00:14.246 T1=10:00:14.247 servidor=10:00:14.227 RTT=0.8 ms
[servidor-B 10:00:14.227]   mejor muestra RTT=0.8 ms -> hora estimada = servidor + RTT/2 (precision +/-0.4 ms)
[servidor-B 10:00:14.227]   antes 10:00:14.247 | despues 10:00:14.227 | ajuste -0.020 s | error real -0.0 ms
[servidor-B 10:00:18.247] === Resincronizacion periodica 4/7 ===
[servidor-B 10:00:18.248]   muestra 1: T0=10:00:18.247 T1=10:00:18.248 servidor=10:00:18.228 RTT=1.2 ms
[servidor-B 10:00:18.249]   muestra 2: T0=10:00:18.248 T1=10:00:18.249 servidor=10:00:18.229 RTT=0.6 ms
[servidor-B 10:00:18.250]   muestra 3: T0=10:00:18.249 T1=10:00:18.250 servidor=10:00:18.229 RTT=0.6 ms
[servidor-B 10:00:18.230]   mejor muestra RTT=0.6 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-B 10:00:18.230]   antes 10:00:18.250 | despues 10:00:18.230 | ajuste -0.020 s | error real -0.0 ms
[servidor-B 10:00:22.252] === Resincronizacion periodica 5/7 ===
[servidor-B 10:00:26.275]   muestra 1: el servidor de tiempo NO responde (TaskCanceledException: The request wa
s canceled due to the configured HttpClient.Timeout of 4 seconds elapsing.)
[servidor-B 10:00:28.885]   muestra 2: el servidor de tiempo NO responde (HttpRequestException: Name or service
not known (time-server:8080))
[servidor-B 10:00:28.887]   muestra 3: el servidor de tiempo NO responde (HttpRequestException: Connection refu
sed (time-server:8080))
[servidor-B 10:00:28.887]   sin muestras validas: se mantiene el reloj (correccion acumulada -6.081 s). Error r
eal actual: +53.0 ms
[servidor-B 10:00:32.906] === Resincronizacion periodica 6/7 ===
[servidor-B 10:00:32.909]   muestra 1: T0=10:00:32.906 T1=10:00:32.909 servidor=10:00:32.835 RTT=2.1 ms
[servidor-B 10:00:32.909]   muestra 2: T0=10:00:32.909 T1=10:00:32.909 servidor=10:00:32.836 RTT=0.8 ms
[servidor-B 10:00:32.910]   muestra 3: T0=10:00:32.909 T1=10:00:32.910 servidor=10:00:32.837 RTT=0.7 ms
[servidor-B 10:00:32.837]   mejor muestra RTT=0.7 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-B 10:00:32.837]   antes 10:00:32.910 | despues 10:00:32.837 | ajuste -0.073 s | error real -0.1 ms
[servidor-B 10:00:36.860] === Resincronizacion periodica 7/7 ===
[servidor-B 10:00:36.861]   muestra 1: T0=10:00:36.860 T1=10:00:36.861 servidor=10:00:36.841 RTT=1.2 ms
[servidor-B 10:00:36.862]   muestra 2: T0=10:00:36.861 T1=10:00:36.862 servidor=10:00:36.842 RTT=0.9 ms
[servidor-B 10:00:36.863]   muestra 3: T0=10:00:36.862 T1=10:00:36.863 servidor=10:00:36.843 RTT=0.8 ms
[servidor-B 10:00:36.843]   mejor muestra RTT=0.8 ms -> hora estimada = servidor + RTT/2 (precision +/-0.4 ms)
[servidor-B 10:00:36.843]   antes 10:00:36.863 | despues 10:00:36.843 | ajuste -0.020 s | error real -0.0 ms
[servidor-B 10:00:36.843] fin

===== client-c =====

[servidor-C 09:59:55.053] iniciado. Hora local 09:59:55.038 | servidor http://time-server:8080 | deriva 0 ppm
[servidor-C 09:59:55.057] diferencia real con el servidor antes de sincronizar: -7.000 s
[servidor-C 09:59:55.150] === Fase 'normal': red Docker sin latencia agregada ===
[servidor-C 09:59:55.189]   muestra 1: T0=09:59:55.152 T1=09:59:55.189 servidor=10:00:02.154 RTT=37.4 ms
[servidor-C 09:59:55.191]   muestra 2: T0=09:59:55.189 T1=09:59:55.191 servidor=10:00:02.190 RTT=1.7 ms
[servidor-C 09:59:55.192]   muestra 3: T0=09:59:55.191 T1=09:59:55.192 servidor=10:00:02.192 RTT=1.0 ms
[servidor-C 10:00:02.196]   mejor muestra RTT=1.0 ms -> hora estimada = servidor + RTT/2 (precision +/-0.5 ms)
[servidor-C 10:00:02.196]   antes 09:59:55.196 | despues 10:00:02.196 | ajuste +7.000 s | error real -0.2 ms
[servidor-C 10:00:06.196] === Resincronizacion periodica 1/7 ===
[servidor-C 10:00:06.197]   muestra 1: T0=10:00:06.196 T1=10:00:06.197 servidor=10:00:06.197 RTT=1.1 ms
[servidor-C 10:00:06.198]   muestra 2: T0=10:00:06.197 T1=10:00:06.198 servidor=10:00:06.198 RTT=0.7 ms
[servidor-C 10:00:06.199]   muestra 3: T0=10:00:06.198 T1=10:00:06.199 servidor=10:00:06.198 RTT=0.6 ms
[servidor-C 10:00:06.199]   mejor muestra RTT=0.6 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-C 10:00:06.199]   antes 10:00:06.199 | despues 10:00:06.199 | ajuste -0.000 s | error real -0.1 ms
[servidor-C 10:00:10.198] === Resincronizacion periodica 2/7 ===
[servidor-C 10:00:10.200]   muestra 1: T0=10:00:10.198 T1=10:00:10.200 servidor=10:00:10.199 RTT=1.1 ms
[servidor-C 10:00:10.201]   muestra 2: T0=10:00:10.200 T1=10:00:10.201 servidor=10:00:10.200 RTT=1.0 ms
[servidor-C 10:00:10.201]   muestra 3: T0=10:00:10.201 T1=10:00:10.201 servidor=10:00:10.201 RTT=0.7 ms
[servidor-C 10:00:10.201]   mejor muestra RTT=0.7 ms -> hora estimada = servidor + RTT/2 (precision +/-0.4 ms)
[servidor-C 10:00:10.201]   antes 10:00:10.201 | despues 10:00:10.201 | ajuste -0.000 s | error real -0.1 ms
[servidor-C 10:00:14.202] === Resincronizacion periodica 3/7 ===
[servidor-C 10:00:14.203]   muestra 1: T0=10:00:14.202 T1=10:00:14.203 servidor=10:00:14.203 RTT=1.1 ms
[servidor-C 10:00:14.204]   muestra 2: T0=10:00:14.203 T1=10:00:14.204 servidor=10:00:14.204 RTT=0.7 ms
[servidor-C 10:00:14.205]   muestra 3: T0=10:00:14.204 T1=10:00:14.205 servidor=10:00:14.204 RTT=0.6 ms
[servidor-C 10:00:14.205]   mejor muestra RTT=0.6 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-C 10:00:14.205]   antes 10:00:14.205 | despues 10:00:14.205 | ajuste +0.000 s | error real -0.0 ms
[servidor-C 10:00:18.205] === Resincronizacion periodica 4/7 ===
[servidor-C 10:00:18.206]   muestra 1: T0=10:00:18.205 T1=10:00:18.206 servidor=10:00:18.205 RTT=1.5 ms
[servidor-C 10:00:18.207]   muestra 2: T0=10:00:18.206 T1=10:00:18.207 servidor=10:00:18.207 RTT=0.8 ms
[servidor-C 10:00:18.208]   muestra 3: T0=10:00:18.207 T1=10:00:18.208 servidor=10:00:18.207 RTT=0.6 ms
[servidor-C 10:00:18.208]   mejor muestra RTT=0.6 ms -> hora estimada = servidor + RTT/2 (precision +/-0.3 ms)
[servidor-C 10:00:18.208]   antes 10:00:18.208 | despues 10:00:18.208 | ajuste +0.000 s | error real -0.0 ms
[servidor-C 10:00:22.209] === Resincronizacion periodica 5/7 ===
[servidor-C 10:00:26.212]   muestra 1: el servidor de tiempo NO responde (TaskCanceledException: The request wa
s canceled due to the configured HttpClient.Timeout of 4 seconds elapsing.)
[servidor-C 10:00:30.210]   muestra 2: T0=10:00:26.212 T1=10:00:30.210 servidor=10:00:30.208 RTT=3998.0 ms  ->
DESCARTADA (RTT mayor al limite)
[servidor-C 10:00:30.212]   muestra 3: T0=10:00:30.210 T1=10:00:30.212 servidor=10:00:30.211 RTT=1.6 ms
```

```
[servidor-C 10:00:30.212]   mejor muestra RTT=1.6 ms -> hora estimada = servidor + RTT/2 (precision +/-0.8 ms)
[servidor-C 10:00:30.212]   antes 10:00:30.212 | despues 10:00:30.212 | ajuste +0.000 s | error real +0.3 ms
[servidor-C 10:00:34.212] === Resincronizacion periodica 6/7 ===
[servidor-C 10:00:34.213]   muestra 1: T0=10:00:34.212 T1=10:00:34.213 servidor=10:00:34.212 RTT=1.8 ms
[servidor-C 10:00:34.214]   muestra 2: T0=10:00:34.213 T1=10:00:34.214 servidor=10:00:34.213 RTT=0.8 ms
[servidor-C 10:00:34.215]   muestra 3: T0=10:00:34.214 T1=10:00:34.215 servidor=10:00:34.214 RTT=0.7 ms
[servidor-C 10:00:34.215]   mejor muestra RTT=0.7 ms -> hora estimada = servidor + RTT/2 (precision +/-0.4 ms)
[servidor-C 10:00:34.215]   antes 10:00:34.215 | despues 10:00:34.215 | ajuste -0.000 s | error real -0.0 ms
[servidor-C 10:00:38.214] === Resincronizacion periodica 7/7 ===
[servidor-C 10:00:38.215]   muestra 1: T0=10:00:38.214 T1=10:00:38.215 servidor=10:00:38.214 RTT=1.3 ms
[servidor-C 10:00:38.216]   muestra 2: T0=10:00:38.215 T1=10:00:38.216 servidor=10:00:38.215 RTT=0.8 ms
[servidor-C 10:00:38.217]   muestra 3: T0=10:00:38.216 T1=10:00:38.217 servidor=10:00:38.216 RTT=0.9 ms
[servidor-C 10:00:38.217]   mejor muestra RTT=0.8 ms -> hora estimada = servidor + RTT/2 (precision +/-0.4 ms)
[servidor-C 10:00:38.217]   antes 10:00:38.217 | despues 10:00:38.217 | ajuste -0.000 s | error real -0.1 ms
[servidor-C 10:00:38.217] fin
```

Ejercicio 2 — Berkeley (7 escenarios)

```
===== Escenario 1 - diferencias iniciales de hasta 5 s =====

Relojes iniciales: coordinador 0 s, nodo-1 5 s, nodo-2 -3 s, nodo-3 2 s

[coordinador 09:59:58.569] iniciado. Hora local 09:59:58.549 | nodos: http://nodo-1:8080, http://nodo-2:8080, h
ttp://nodo-3:8080 | intervalo 5 s | promedio simple
[coordinador 10:00:00.580] ===== Ronda 1 =====
[coordinador 10:00:00.782] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:00.782] coordinador   10:00:00.775   +0.000 s     si           +1.024 s     10:00:01.799
[coordinador 10:00:00.782] nodo-1        10:00:05.813   +5.038 s     si           -4.014 s     10:00:01.799
[coordinador 10:00:00.783] nodo-2        09:59:57.806   -2.969 s     si           +3.994 s     10:00:01.799
[coordinador 10:00:00.783] nodo-3        10:00:02.804   +2.029 s     si           -1.004 s     10:00:01.799
[coordinador 10:00:00.786] promedio de diferencias: +1.024 s | dispersion antes del ajuste (max - min): +8.008
s
[coordinador 10:00:06.858] ===== Ronda 2 =====
[coordinador 10:00:06.859] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:06.859] coordinador   10:00:06.859   +0.000 s     si           -0.022 s     10:00:06.838
[coordinador 10:00:06.860] nodo-1        10:00:06.821   -0.038 s     si           +0.017 s     10:00:06.838
[coordinador 10:00:06.860] nodo-2        10:00:06.840   -0.019 s     si           -0.002 s     10:00:06.838
[coordinador 10:00:06.860] nodo-3        10:00:06.830   -0.029 s     si           +0.007 s     10:00:06.838
[coordinador 10:00:06.860] promedio de diferencias: -0.022 s | dispersion antes del ajuste (max - min): +0.038
s
[coordinador 10:00:06.844] fin

[nodo-1] nodo iniciado, hora local 10:00:03.190 (deriva 0 ppm)
[nodo-1] ajuste recibido -4.014 s: 10:00:05.806 -> 10:00:01.792
[nodo-1] ajuste recibido +0.017 s: 10:00:06.823 -> 10:00:06.839
[nodo-2] nodo iniciado, hora local 09:59:55.436 (deriva 2000 ppm)
[nodo-2] ajuste recibido +3.994 s: 09:59:57.822 -> 10:00:01.816
[nodo-2] ajuste recibido -0.002 s: 10:00:06.844 -> 10:00:06.841
[nodo-3] nodo iniciado, hora local 10:00:00.494 (deriva 0 ppm)
[nodo-3] ajuste recibido -1.004 s: 10:00:02.833 -> 10:00:01.829
[nodo-3] ajuste recibido +0.007 s: 10:00:06.836 -> 10:00:06.843

===== Escenario 2 - diferencias iniciales de hasta 10 s =====

Relojes iniciales: coordinador 0 s, nodo-1 10 s, nodo-2 -6 s, nodo-3 4 s

[coordinador 09:59:58.646] iniciado. Hora local 09:59:58.634 | nodos: http://nodo-1:8080, http://nodo-2:8080, h
ttp://nodo-3:8080 | intervalo 5 s | promedio simple
[coordinador 10:00:00.655] ===== Ronda 1 =====
[coordinador 10:00:00.879] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:00.879] coordinador   10:00:00.871   +0.000 s     si           +2.029 s     10:00:02.900
[coordinador 10:00:00.879] nodo-1        10:00:10.918   +10.047 s    si           -8.018 s     10:00:02.900
[coordinador 10:00:00.879] nodo-2        09:59:54.909   -5.962 s     si           +7.992 s     10:00:02.900
[coordinador 10:00:00.879] nodo-3        10:00:04.903   +4.032 s     si           -2.003 s     10:00:02.900
[coordinador 10:00:00.884] promedio de diferencias: +2.029 s | dispersion antes del ajuste (max - min): +16.010
s
[coordinador 10:00:07.971] ===== Ronda 2 =====
[coordinador 10:00:07.973] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:07.974] coordinador   10:00:07.973   +0.000 s     si           -0.026 s     10:00:07.947
[coordinador 10:00:07.974] nodo-1        10:00:07.926   -0.047 s     si           +0.021 s     10:00:07.947
```

```
[coordinador 10:00:07.974] nodo-2        10:00:07.947   -0.026 s     si           -0.000 s     10:00:07.947
[coordinador 10:00:07.974] nodo-3        10:00:07.941   -0.032 s     si           +0.006 s     10:00:07.947
[coordinador 10:00:07.974] promedio de diferencias: -0.026 s | dispersion antes del ajuste (max - min): +0.047
s
[coordinador 10:00:07.952] fin

[nodo-1] nodo iniciado, hora local 10:00:08.200 (deriva 0 ppm)
[nodo-1] ajuste recibido -8.018 s: 10:00:10.908 -> 10:00:02.890
[nodo-1] ajuste recibido +0.021 s: 10:00:07.927 -> 10:00:07.948
[nodo-2] nodo iniciado, hora local 09:59:52.528 (deriva 2000 ppm)
[nodo-2] ajuste recibido +7.992 s: 09:59:54.928 -> 10:00:02.920
[nodo-2] ajuste recibido -0.000 s: 10:00:07.951 -> 10:00:07.951
[nodo-3] nodo iniciado, hora local 10:00:02.513 (deriva 0 ppm)
[nodo-3] ajuste recibido -2.003 s: 10:00:04.942 -> 10:00:02.939
[nodo-3] ajuste recibido +0.006 s: 10:00:07.946 -> 10:00:07.952

===== Escenario 3 - diferencias iniciales de hasta 20 s =====

Relojes iniciales: coordinador 0 s, nodo-1 20 s, nodo-2 -12 s, nodo-3 8 s

[coordinador 09:59:58.608] iniciado. Hora local 09:59:58.600 | nodos: http://nodo-1:8080, http://nodo-2:8080, h
ttp://nodo-3:8080 | intervalo 5 s | promedio simple
[coordinador 10:00:00.618] ===== Ronda 1 =====
[coordinador 10:00:00.835] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:00.835] coordinador   10:00:00.828   +0.000 s     si           +4.031 s     10:00:04.859
[coordinador 10:00:00.836] nodo-1        10:00:20.873   +20.045 s    si           -16.013 s    10:00:04.859
[coordinador 10:00:00.836] nodo-2        09:59:48.872   -11.956 s    si           +15.987 s    10:00:04.859
[coordinador 10:00:00.836] nodo-3        10:00:08.864   +8.036 s     si           -4.005 s     10:00:04.859
[coordinador 10:00:00.839] promedio de diferencias: +4.031 s | dispersion antes del ajuste (max - min): +32.000
s
[coordinador 10:00:09.918] ===== Ronda 2 =====
[coordinador 10:00:09.920] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:09.920] coordinador   10:00:09.920   +0.000 s     si           -0.028 s     10:00:09.892
[coordinador 10:00:09.920] nodo-1        10:00:09.876   -0.045 s     si           +0.016 s     10:00:09.892
[coordinador 10:00:09.920] nodo-2        10:00:09.888   -0.032 s     si           +0.004 s     10:00:09.892
[coordinador 10:00:09.920] nodo-3        10:00:09.884   -0.036 s     si           +0.008 s     10:00:09.892
[coordinador 10:00:09.920] promedio de diferencias: -0.028 s | dispersion antes del ajuste (max - min): +0.045
s
[coordinador 10:00:09.896] fin

[nodo-1] nodo iniciado, hora local 10:00:18.244 (deriva 0 ppm)
[nodo-1] ajuste recibido -16.013 s: 10:00:20.859 -> 10:00:04.846
[nodo-1] ajuste recibido +0.016 s: 10:00:09.877 -> 10:00:09.893
[nodo-2] nodo iniciado, hora local 09:59:46.528 (deriva 2000 ppm)
[nodo-2] ajuste recibido +15.987 s: 09:59:48.875 -> 10:00:04.862
[nodo-2] ajuste recibido +0.004 s: 10:00:09.890 -> 10:00:09.894
[nodo-3] nodo iniciado, hora local 10:00:06.497 (deriva 0 ppm)
[nodo-3] ajuste recibido -4.005 s: 10:00:08.886 -> 10:00:04.881
[nodo-3] ajuste recibido +0.008 s: 10:00:09.888 -> 10:00:09.895

===== Escenario 4 - nodo-3 desfasado 5 minutos, promedio simple =====

Relojes iniciales: coordinador 0 s, nodo-1 5 s, nodo-2 -3 s, nodo-3 300 s

[coordinador 09:59:58.620] iniciado. Hora local 09:59:58.612 | nodos: http://nodo-1:8080, http://nodo-2:8080, h
ttp://nodo-3:8080 | intervalo 5 s | promedio simple
[coordinador 10:00:00.635] ===== Ronda 1 =====
[coordinador 10:00:00.874] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:00.874] coordinador   10:00:00.866   +0.000 s     si           +75.538 s    10:01:16.404
[coordinador 10:00:00.874] nodo-1        10:00:05.922   +5.056 s     si           +70.482 s    10:01:16.404
[coordinador 10:00:00.874] nodo-2        09:59:57.918   -2.948 s     si           +78.486 s    10:01:16.404
[coordinador 10:00:00.874] nodo-3        10:05:00.910   +300.043 s   si           -224.505 s   10:01:16.404
[coordinador 10:00:00.878] promedio de diferencias: +75.538 s | dispersion antes del ajuste (max - min): +302.9
92 s
[coordinador 10:01:21.464] ===== Ronda 2 =====
[coordinador 10:01:21.466] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:01:21.466] coordinador   10:01:21.466   +0.000 s     si           -0.035 s     10:01:21.431
[coordinador 10:01:21.466] nodo-1        10:01:21.410   -0.056 s     si           +0.021 s     10:01:21.431
[coordinador 10:01:21.466] nodo-2        10:01:21.426   -0.040 s     si           +0.005 s     10:01:21.431
[coordinador 10:01:21.466] nodo-3        10:01:21.423   -0.043 s     si           +0.008 s     10:01:21.431
[coordinador 10:01:21.466] promedio de diferencias: -0.035 s | dispersion antes del ajuste (max - min): +0.056
s
[coordinador 10:01:21.436] fin
```

```
[nodo-1] nodo iniciado, hora local 10:00:03.232 (deriva 0 ppm)
[nodo-1] ajuste recibido +70.482 s: 10:00:05.899 -> 10:01:16.380
[nodo-1] ajuste recibido +0.021 s: 10:01:21.411 -> 10:01:21.432
[nodo-2] nodo iniciado, hora local 09:59:55.504 (deriva 2000 ppm)
[nodo-2] ajuste recibido +78.486 s: 09:59:57.914 -> 10:01:16.401
[nodo-2] ajuste recibido +0.005 s: 10:01:21.429 -> 10:01:21.434
[nodo-3] nodo iniciado, hora local 10:04:58.492 (deriva 0 ppm)
[nodo-3] ajuste recibido -224.505 s: 10:05:00.926 -> 10:01:16.420
[nodo-3] ajuste recibido +0.008 s: 10:01:21.427 -> 10:01:21.435

===== Escenario 5 - nodo-3 desfasado 5 minutos, promedio tolerante a fallas (umbral 30 s) =====

Relojes iniciales: coordinador 0 s, nodo-1 5 s, nodo-2 -3 s, nodo-3 300 s

[coordinador 09:59:58.735] iniciado. Hora local 09:59:58.717 | nodos: http://nodo-1:8080, http://nodo-2:8080, h
ttp://nodo-3:8080 | intervalo 5 s | promedio tolerante a fallas (descarta relojes a mas de 30 s de la mediana)
[coordinador 10:00:00.747] ===== Ronda 1 =====
[coordinador 10:00:00.939] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:00.939] coordinador   10:00:00.932   +0.000 s     si           +0.696 s     10:00:01.628
[coordinador 10:00:00.939] nodo-1        10:00:05.973   +5.041 s     si           -4.345 s     10:00:01.628
[coordinador 10:00:00.939] nodo-2        09:59:57.979   -2.954 s     si           +3.649 s     10:00:01.628
[coordinador 10:00:00.939] nodo-3        10:05:00.977   +300.045 s   NO (atipico) -299.349 s   10:00:01.628
[coordinador 10:00:00.943] promedio de diferencias: +0.696 s | dispersion antes del ajuste (max - min): +302.99
8 s
[coordinador 10:00:06.683] ===== Ronda 2 =====
[coordinador 10:00:06.685] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:06.685] coordinador   10:00:06.685   +0.000 s     si           -0.030 s     10:00:06.655
[coordinador 10:00:06.685] nodo-1        10:00:06.644   -0.041 s     si           +0.011 s     10:00:06.655
[coordinador 10:00:06.685] nodo-2        10:00:06.651   -0.035 s     si           +0.004 s     10:00:06.655
[coordinador 10:00:06.685] nodo-3        10:00:06.641   -0.045 s     si           +0.015 s     10:00:06.655
[coordinador 10:00:06.685] promedio de diferencias: -0.030 s | dispersion antes del ajuste (max - min): +0.045
s
[coordinador 10:00:06.660] fin

[nodo-1] nodo iniciado, hora local 10:00:03.199 (deriva 0 ppm)
[nodo-1] ajuste recibido -4.345 s: 10:00:05.962 -> 10:00:01.616
[nodo-1] ajuste recibido +0.011 s: 10:00:06.645 -> 10:00:06.656
[nodo-2] nodo iniciado, hora local 09:59:55.463 (deriva 2000 ppm)
[nodo-2] ajuste recibido +3.649 s: 09:59:57.977 -> 10:00:01.627
[nodo-2] ajuste recibido +0.004 s: 10:00:06.653 -> 10:00:06.658
[nodo-3] nodo iniciado, hora local 10:04:58.486 (deriva 0 ppm)
[nodo-3] ajuste recibido -299.349 s: 10:05:00.988 -> 10:00:01.639
[nodo-3] ajuste recibido +0.015 s: 10:00:06.644 -> 10:00:06.659

===== Escenario 6 - nodo-3 se desconecta despues de la ronda 1 y vuelve antes de la ronda 4 =====

Relojes iniciales: coordinador 0 s, nodo-1 10 s, nodo-2 -6 s, nodo-3 4 s
nodo-3 detenido tras la ronda 1 (docker compose stop) y reiniciado tras la ronda 3 (vuelve con su reloj origina
l)

[coordinador 09:59:58.855] iniciado. Hora local 09:59:58.845 | nodos: http://nodo-1:8080, http://nodo-2:8080, h
ttp://nodo-3:8080 | intervalo 5 s | promedio simple
[coordinador 10:00:00.866] ===== Ronda 1 =====
[coordinador 10:00:01.081] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:01.082] coordinador   10:00:01.074   +0.000 s     si           +2.039 s     10:00:03.113
[coordinador 10:00:01.082] nodo-1        10:00:11.136   +10.061 s    si           -8.023 s     10:00:03.113
[coordinador 10:00:01.082] nodo-2        09:59:55.116   -5.958 s     si           +7.997 s     10:00:03.113
[coordinador 10:00:01.082] nodo-3        10:00:05.126   +4.052 s     si           -2.013 s     10:00:03.113
[coordinador 10:00:01.086] promedio de diferencias: +2.039 s | dispersion antes del ajuste (max - min): +16.019
s
[coordinador 10:00:08.171] ===== Ronda 2 =====
[coordinador 10:00:10.175] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:10.175] coordinador   10:00:10.174   +0.000 s     si           -0.030 s     10:00:10.144
[coordinador 10:00:10.175] nodo-1        10:00:10.113   -0.061 s     si           +0.031 s     10:00:10.144
[coordinador 10:00:10.175] nodo-2        10:00:10.145   -0.030 s     si           -0.001 s     10:00:10.144
[coordinador 10:00:10.175] nodo-3        SIN RESPUESTA  -            no           -            -  (timeout)
[coordinador 10:00:10.175] promedio de diferencias: -0.030 s | dispersion antes del ajuste (max - min): +0.061
s
[coordinador 10:00:15.145] ===== Ronda 3 =====
[coordinador 10:00:17.146] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:17.146] coordinador   10:00:17.146   +0.000 s     si           +0.005 s     10:00:17.151
[coordinador 10:00:17.146] nodo-1        10:00:17.146   +0.000 s     si           +0.004 s     10:00:17.151
```

```
[coordinador 10:00:17.146] nodo-2        10:00:17.160   +0.014 s     si           -0.009 s     10:00:17.151
[coordinador 10:00:17.146] nodo-3        SIN RESPUESTA  -            no           -            -  (timeout)
[coordinador 10:00:17.146] promedio de diferencias: +0.005 s | dispersion antes del ajuste (max - min): +0.014
s
[coordinador 10:00:22.156] ===== Ronda 4 =====
[coordinador 10:00:22.253] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:22.253] coordinador   10:00:22.253   +0.000 s     si           +0.509 s     10:00:22.762
[coordinador 10:00:22.253] nodo-1        10:00:22.252   -0.000 s     si           +0.510 s     10:00:22.762
[coordinador 10:00:22.253] nodo-2        10:00:22.267   +0.014 s     si           +0.496 s     10:00:22.762
[coordinador 10:00:22.253] nodo-3        10:00:24.277   +2.024 s     si           -1.515 s     10:00:22.762
[coordinador 10:00:22.253] promedio de diferencias: +0.509 s | dispersion antes del ajuste (max - min): +2.024
s
[coordinador 10:00:22.778] fin

[nodo-1] nodo iniciado, hora local 10:00:08.461 (deriva 0 ppm)
[nodo-1] ajuste recibido -8.023 s: 10:00:11.105 -> 10:00:03.083
[nodo-1] ajuste recibido +0.031 s: 10:00:10.115 -> 10:00:10.146
[nodo-1] ajuste recibido +0.004 s: 10:00:17.147 -> 10:00:17.151
[nodo-1] ajuste recibido +0.510 s: 10:00:22.253 -> 10:00:22.763
[nodo-2] nodo iniciado, hora local 09:59:52.688 (deriva 2000 ppm)
[nodo-2] ajuste recibido +7.997 s: 09:59:55.121 -> 10:00:03.118
[nodo-2] ajuste recibido -0.001 s: 10:00:10.152 -> 10:00:10.151
[nodo-2] ajuste recibido -0.009 s: 10:00:17.166 -> 10:00:17.157
[nodo-2] ajuste recibido +0.496 s: 10:00:22.269 -> 10:00:22.764
[nodo-3] nodo iniciado, hora local 10:00:02.734 (deriva 0 ppm)
[nodo-3] ajuste recibido -2.013 s: 10:00:05.132 -> 10:00:03.119
[nodo-3] nodo iniciado, hora local 10:00:19.812 (deriva 0 ppm)
[nodo-3] ajuste recibido -1.515 s: 10:00:24.254 -> 10:00:22.739

===== Escenario 7 - falla el coordinador (nodo-2 deriva 20000 ppm = 20 ms/s) =====

Relojes iniciales: coordinador 0 s, nodo-1 10 s, nodo-2 -6 s, nodo-3 4 s

Con coordinador (tras la ronda 3)  nodo-1 10:00:14.756 (+0 ms) | nodo-2 10:00:14.795 (+40 ms) | nodo-3 10:00:1
4.765 (+9 ms) | dispersion 40 ms
--- se detiene el coordinador ---
Sin coordinador, +5 s              nodo-1 10:00:20.566 (+0 ms) | nodo-2 10:00:20.719 (+154 ms) | nodo-3 10:00:2
0.572 (+7 ms) | dispersion 154 ms
Sin coordinador, +10 s             nodo-1 10:00:25.801 (+0 ms) | nodo-2 10:00:26.073 (+272 ms) | nodo-3 10:00:2
5.821 (+19 ms) | dispersion 272 ms
Sin coordinador, +20 s             nodo-1 10:00:36.100 (+0 ms) | nodo-2 10:00:36.578 (+478 ms) | nodo-3 10:00:3
6.119 (+19 ms) | dispersion 478 ms

[coordinador 09:59:58.614] iniciado. Hora local 09:59:58.604 | nodos: http://nodo-1:8080, http://nodo-2:8080, h
ttp://nodo-3:8080 | intervalo 5 s | promedio simple
[coordinador 10:00:00.627] ===== Ronda 1 =====
[coordinador 10:00:00.920] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:00.921] coordinador   10:00:00.913   +0.000 s     si           +2.040 s     10:00:02.953
[coordinador 10:00:00.921] nodo-1        10:00:10.980   +10.067 s    si           -8.027 s     10:00:02.953
[coordinador 10:00:00.921] nodo-2        09:59:54.965   -5.948 s     si           +7.988 s     10:00:02.953
[coordinador 10:00:00.921] nodo-3        10:00:04.952   +4.039 s     si           -2.000 s     10:00:02.953
[coordinador 10:00:00.925] promedio de diferencias: +2.040 s | dispersion antes del ajuste (max - min): +16.015
s
[coordinador 10:00:08.009] ===== Ronda 2 =====
[coordinador 10:00:08.010] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:08.011] coordinador   10:00:08.010   +0.000 s     si           -0.010 s     10:00:08.001
[coordinador 10:00:08.011] nodo-1        10:00:07.943   -0.067 s     si           +0.057 s     10:00:08.001
[coordinador 10:00:08.011] nodo-2        10:00:08.078   +0.067 s     si           -0.077 s     10:00:08.001
[coordinador 10:00:08.011] nodo-3        10:00:07.971   -0.039 s     si           +0.030 s     10:00:08.001
[coordinador 10:00:08.011] promedio de diferencias: -0.010 s | dispersion antes del ajuste (max - min): +0.134
s
[coordinador 10:00:13.006] ===== Ronda 3 =====
[coordinador 10:00:13.007] participante  hora           diferencia   en promedio  ajuste       hora ajustada
[coordinador 10:00:13.007] coordinador   10:00:13.007   +0.000 s     si           +0.025 s     10:00:13.032
[coordinador 10:00:13.007] nodo-1        10:00:13.007   +0.000 s     si           +0.025 s     10:00:13.032
[coordinador 10:00:13.007] nodo-2        10:00:13.108   +0.100 s     si           -0.075 s     10:00:13.032
[coordinador 10:00:13.007] nodo-3        10:00:13.007   +0.000 s     si           +0.025 s     10:00:13.032
[coordinador 10:00:13.007] promedio de diferencias: +0.025 s | dispersion antes del ajuste (max - min): +0.100
s

[nodo-1] nodo iniciado, hora local 10:00:08.277 (deriva 0 ppm)
[nodo-1] ajuste recibido -8.027 s: 10:00:10.944 -> 10:00:02.917
[nodo-1] ajuste recibido +0.057 s: 10:00:07.944 -> 10:00:08.002
```

```
[nodo-1] ajuste recibido +0.025 s: 10:00:13.008 -> 10:00:13.033
[nodo-2] nodo iniciado, hora local 09:59:52.475 (deriva 20000 ppm)
[nodo-2] ajuste recibido +7.988 s: 09:59:54.977 -> 10:00:02.964
[nodo-2] ajuste recibido -0.077 s: 10:00:08.080 -> 10:00:08.003
[nodo-2] ajuste recibido -0.075 s: 10:00:13.109 -> 10:00:13.034
[nodo-3] nodo iniciado, hora local 10:00:02.521 (deriva 0 ppm)
[nodo-3] ajuste recibido -2.000 s: 10:00:04.970 -> 10:00:02.971
[nodo-3] ajuste recibido +0.030 s: 10:00:07.975 -> 10:00:08.004
[nodo-3] ajuste recibido +0.025 s: 10:00:13.009 -> 10:00:13.034
```

Ejercicio 3 — Lamport

```
Escenario: hora simulada 10:00:00 = 2026-11-11T20:37:27.295Z (UTC real)
Relojes fisicos: OrdersService +0 s | InventoryService -3 s | PaymentsService +5 s

===== Pedido 1: P001 x 2 (hay stock) =====

$ curl -X POST http://localhost:8090/api/orders -d '{"productId":"P001","quantity":2,"amount":150}'
{"orderId":"ORD-001","status":"CONFIRMED","lamport":12} [HTTP 200]

===== Pedido 2: P003 x 1 (sin stock) =====

$ curl -X POST http://localhost:8090/api/orders -d '{"productId":"P003","quantity":1,"amount":80}'
{"orderId":"ORD-002","status":"REJECTED"} [HTTP 409]

===== Registro de cada servicio =====

[OrdersService] iniciado, reloj fisico 09:59:58.447
[OrdersService] fisico 10:00:00.146 | Lamport   1 | vector [I=0,O=1,P=0] | Pedido recibido ORD-001: 2 x P001
[OrdersService] fisico 10:00:00.151 | Lamport   2 | vector [I=0,O=2,P=0] | Mensaje enviado a InventoryService
(pedido ORD-001)
[OrdersService] fisico 10:00:00.309 | Lamport   6 | vector [I=3,O=3,P=0] | Respuesta de InventoryService recibi
da (pedido ORD-001) (recibido L=5 > local L=2: salta a 6)
[OrdersService] fisico 10:00:00.316 | Lamport   7 | vector [I=3,O=4,P=0] | Solicitud de pago enviada a Payments
Service (pedido ORD-001)
[OrdersService] fisico 10:00:00.481 | Lamport  11 | vector [I=3,O=5,P=7] | Confirmacion de pago recibida (pedid
o ORD-001) (recibido L=10 > local L=7: salta a 11)
[OrdersService] fisico 10:00:00.482 | Lamport  12 | vector [I=3,O=6,P=7] | Pedido ORD-001 confirmado
[OrdersService] fisico 10:00:00.529 | Lamport  13 | vector [I=3,O=7,P=7] | Pedido recibido ORD-002: 1 x P003
[OrdersService] fisico 10:00:00.529 | Lamport  14 | vector [I=3,O=8,P=7] | Mensaje enviado a InventoryService
(pedido ORD-002)
[OrdersService] fisico 10:00:00.534 | Lamport  18 | vector [I=6,O=9,P=7] | Respuesta de InventoryService recibi
da (pedido ORD-002) (recibido L=17 > local L=14: salta a 18)
[OrdersService] fisico 10:00:00.534 | Lamport  19 | vector [I=6,O=10,P=7] | Pedido ORD-002 rechazado: sin stock

[InventoryService] iniciado, reloj fisico 09:59:55.244
[InventoryService] fisico 09:59:57.290 | Lamport   3 | vector [I=1,O=2,P=0] | Mensaje recibido de OrdersService
(pedido ORD-001) (recibido L=2 > local L=0: salta a 3)
[InventoryService] fisico 09:59:57.292 | Lamport   4 | vector [I=2,O=2,P=0] | Inventario verificado: P001 stock
10, pedido 2 -> reservado
[InventoryService] fisico 09:59:57.292 | Lamport   5 | vector [I=3,O=2,P=0] | Respuesta enviada a OrdersService
(pedido ORD-001)
[InventoryService] fisico 09:59:57.533 | Lamport  15 | vector [I=4,O=8,P=7] | Mensaje recibido de OrdersService
(pedido ORD-002) (recibido L=14 > local L=5: salta a 15)
[InventoryService] fisico 09:59:57.533 | Lamport  16 | vector [I=5,O=8,P=7] | Inventario verificado: P003 stock
0, pedido 1 -> sin stock
[InventoryService] fisico 09:59:57.533 | Lamport  17 | vector [I=6,O=8,P=7] | Respuesta enviada a OrdersService
(pedido ORD-002)

[PaymentsService] fisico 10:00:03.162 | Lamport   1 | vector [I=0,O=0,P=1] | Pago de otro pedido procesado (#1,
sin relacion con este flujo)
[PaymentsService] fisico 10:00:03.181 | Lamport   2 | vector [I=0,O=0,P=2] | Pago de otro pedido procesado (#2,
sin relacion con este flujo)
[PaymentsService] fisico 10:00:03.182 | Lamport   3 | vector [I=0,O=0,P=3] | Pago de otro pedido procesado (#3,
sin relacion con este flujo)
[PaymentsService] fisico 10:00:03.182 | Lamport   4 | vector [I=0,O=0,P=4] | Pago de otro pedido procesado (#4,
sin relacion con este flujo)
[PaymentsService] iniciado, reloj fisico 10:00:03.191
[PaymentsService] fisico 10:00:05.419 | Lamport   8 | vector [I=3,O=4,P=5] | Solicitud de pago recibida (pedido
ORD-001) (recibido L=7 > local L=4: salta a 8)
```

```
[PaymentsService] fisico 10:00:05.470 | Lamport   9 | vector [I=3,O=4,P=6] | Pago procesado: $150.00 aprobado
[PaymentsService] fisico 10:00:05.470 | Lamport  10 | vector [I=3,O=4,P=7] | Confirmacion enviada a OrdersServi
ce (pedido ORD-001)


===== GET /api/timeline (eventos de los tres servicios) =====

=== Orden total por reloj de Lamport (desempate por nombre de proceso) ===
proceso           reloj fisico   Lamport  vector          evento
OrdersService     10:00:00.146   1        I=0,O=1,P=0     Pedido recibido ORD-001: 2 x P001
PaymentsService   10:00:03.162   1        I=0,O=0,P=1     Pago de otro pedido procesado (#1, sin relacion con e
ste flujo)
OrdersService     10:00:00.151   2        I=0,O=2,P=0     Mensaje enviado a InventoryService (pedido ORD-001)
PaymentsService   10:00:03.181   2        I=0,O=0,P=2     Pago de otro pedido procesado (#2, sin relacion con e
ste flujo)
InventoryService  09:59:57.290   3        I=1,O=2,P=0     Mensaje recibido de OrdersService (pedido ORD-001) (r
ecibido L=2 > local L=0: salta a 3)
PaymentsService   10:00:03.182   3        I=0,O=0,P=3     Pago de otro pedido procesado (#3, sin relacion con e
ste flujo)
InventoryService  09:59:57.292   4        I=2,O=2,P=0     Inventario verificado: P001 stock 10, pedido 2 -> res
ervado
PaymentsService   10:00:03.182   4        I=0,O=0,P=4     Pago de otro pedido procesado (#4, sin relacion con e
ste flujo)
InventoryService  09:59:57.292   5        I=3,O=2,P=0     Respuesta enviada a OrdersService (pedido ORD-001)
OrdersService     10:00:00.309   6        I=3,O=3,P=0     Respuesta de InventoryService recibida (pedido ORD-00
1) (recibido L=5 > local L=2: salta a 6)
OrdersService     10:00:00.316   7        I=3,O=4,P=0     Solicitud de pago enviada a PaymentsService (pedido O
RD-001)
PaymentsService   10:00:05.419   8        I=3,O=4,P=5     Solicitud de pago recibida (pedido ORD-001) (recibido
L=7 > local L=4: salta a 8)
PaymentsService   10:00:05.470   9        I=3,O=4,P=6     Pago procesado: $150.00 aprobado
PaymentsService   10:00:05.470   10       I=3,O=4,P=7     Confirmacion enviada a OrdersService (pedido ORD-001)
OrdersService     10:00:00.481   11       I=3,O=5,P=7     Confirmacion de pago recibida (pedido ORD-001) (recib
ido L=10 > local L=7: salta a 11)
OrdersService     10:00:00.482   12       I=3,O=6,P=7     Pedido ORD-001 confirmado
OrdersService     10:00:00.529   13       I=3,O=7,P=7     Pedido recibido ORD-002: 1 x P003
OrdersService     10:00:00.529   14       I=3,O=8,P=7     Mensaje enviado a InventoryService (pedido ORD-002)
InventoryService  09:59:57.533   15       I=4,O=8,P=7     Mensaje recibido de OrdersService (pedido ORD-002) (r
ecibido L=14 > local L=5: salta a 15)
InventoryService  09:59:57.533   16       I=5,O=8,P=7     Inventario verificado: P003 stock 0, pedido 1 -> sin
stock
InventoryService  09:59:57.533   17       I=6,O=8,P=7     Respuesta enviada a OrdersService (pedido ORD-002)
OrdersService     10:00:00.534   18       I=6,O=9,P=7     Respuesta de InventoryService recibida (pedido ORD-00
2) (recibido L=17 > local L=14: salta a...
OrdersService     10:00:00.534   19       I=6,O=10,P=7    Pedido ORD-002 rechazado: sin stock

=== Orden por reloj fisico (hora local de cada servicio) ===
proceso           reloj fisico   Lamport  vector          evento
InventoryService  09:59:57.290   3        I=1,O=2,P=0     Mensaje recibido de OrdersService (pedido ORD-001) (r
ecibido L=2 > local L=0: salta a 3)
InventoryService  09:59:57.292   4        I=2,O=2,P=0     Inventario verificado: P001 stock 10, pedido 2 -> res
ervado
InventoryService  09:59:57.292   5        I=3,O=2,P=0     Respuesta enviada a OrdersService (pedido ORD-001)
InventoryService  09:59:57.533   15       I=4,O=8,P=7     Mensaje recibido de OrdersService (pedido ORD-002) (r
ecibido L=14 > local L=5: salta a 15)
InventoryService  09:59:57.533   16       I=5,O=8,P=7     Inventario verificado: P003 stock 0, pedido 1 -> sin
stock
InventoryService  09:59:57.533   17       I=6,O=8,P=7     Respuesta enviada a OrdersService (pedido ORD-002)
OrdersService     10:00:00.146   1        I=0,O=1,P=0     Pedido recibido ORD-001: 2 x P001
OrdersService     10:00:00.151   2        I=0,O=2,P=0     Mensaje enviado a InventoryService (pedido ORD-001)
OrdersService     10:00:00.309   6        I=3,O=3,P=0     Respuesta de InventoryService recibida (pedido ORD-00
1) (recibido L=5 > local L=2: salta a 6)
OrdersService     10:00:00.316   7        I=3,O=4,P=0     Solicitud de pago enviada a PaymentsService (pedido O
RD-001)
OrdersService     10:00:00.481   11       I=3,O=5,P=7     Confirmacion de pago recibida (pedido ORD-001) (recib
ido L=10 > local L=7: salta a 11)
OrdersService     10:00:00.482   12       I=3,O=6,P=7     Pedido ORD-001 confirmado
OrdersService     10:00:00.529   13       I=3,O=7,P=7     Pedido recibido ORD-002: 1 x P003
OrdersService     10:00:00.529   14       I=3,O=8,P=7     Mensaje enviado a InventoryService (pedido ORD-002)
OrdersService     10:00:00.534   18       I=6,O=9,P=7     Respuesta de InventoryService recibida (pedido ORD-00
2) (recibido L=17 > local L=14: salta a...
OrdersService     10:00:00.534   19       I=6,O=10,P=7    Pedido ORD-002 rechazado: sin stock
PaymentsService   10:00:03.162   1        I=0,O=0,P=1     Pago de otro pedido procesado (#1, sin relacion con e
```

```
ste flujo)
PaymentsService   10:00:03.181   2        I=0,O=0,P=2     Pago de otro pedido procesado (#2, sin relacion con e
ste flujo)
PaymentsService   10:00:03.182   3        I=0,O=0,P=3     Pago de otro pedido procesado (#3, sin relacion con e
ste flujo)
PaymentsService   10:00:03.182   4        I=0,O=0,P=4     Pago de otro pedido procesado (#4, sin relacion con e
ste flujo)
PaymentsService   10:00:05.419   8        I=3,O=4,P=5     Solicitud de pago recibida (pedido ORD-001) (recibido
L=7 > local L=4: salta a 8)
PaymentsService   10:00:05.470   9        I=3,O=4,P=6     Pago procesado: $150.00 aprobado
PaymentsService   10:00:05.470   10       I=3,O=4,P=7     Confirmacion enviada a OrdersService (pedido ORD-001)

=== Inconsistencias del reloj fisico: 93 pares donde a -> b pero hora(b) < hora(a) ===
  Orders L=2 [I=0,O=2,P=0] "Mensaje enviado a InventoryService ..."  ->  Inventory L=3 [I=1,O=2,P=0] "Mensaje r
ecibido de OrdersService (..."  (hora(b) - hora(a) = -2.861 s)
  Payments L=10 [I=3,O=4,P=7] "Confirmacion enviada a OrdersServic..."  ->  Orders L=11 [I=3,O=5,P=7] "Confirma
cion de pago recibida (pedi..."  (hora(b) - hora(a) = -4.990 s)
  Orders L=14 [I=3,O=8,P=7] "Mensaje enviado a InventoryService ..."  ->  Inventory L=15 [I=4,O=8,P=7] "Mensaje
recibido de OrdersService (..."  (hora(b) - hora(a) = -2.997 s)

=== Eventos concurrentes con L(a) < L(b): 24 pares (Lamport no los distingue) ===
  Orders L=1 [I=0,O=1,P=0] "Pedido recibido ORD-001: 2 x P001"  ||  Payments L=2 [I=0,O=0,P=2] "Pago de otro pe
dido procesado (#2, ..."
  Orders L=1 [I=0,O=1,P=0] "Pedido recibido ORD-001: 2 x P001"  ||  Payments L=3 [I=0,O=0,P=3] "Pago de otro pe
dido procesado (#3, ..."
  Orders L=1 [I=0,O=1,P=0] "Pedido recibido ORD-001: 2 x P001"  ||  Payments L=4 [I=0,O=0,P=4] "Pago de otro pe
dido procesado (#4, ..."
  Orders L=2 [I=0,O=2,P=0] "Mensaje enviado a InventoryService ..."  ||  Payments L=3 [I=0,O=0,P=3] "Pago de ot
ro pedido procesado (#3, ..."
```

Anexo B — Código fuente principal

src/Clocks/SimulatedClock.cs

```
using System.Globalization;

namespace Clocks;

/// <summary>
/// Application-level clock. It never touches the operating system clock: it is the real
/// UTC time plus an initial offset, a drift (ppm) and the corrections applied by the
/// synchronization algorithms.
///
/// All containers share the host kernel clock, so the skew is simulated here.
/// When SCENARIO_START_UTC is set, that real instant is shown as SCENARIO_BASE_TIME
/// (10:00:00 by default), so the logs match the times of the case study.
/// </summary>
public sealed class SimulatedClock
{
    private static readonly DateTime BaseDay = new(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc);

    private readonly DateTime _startUtc;
    private readonly DateTime _shownAtStart;
    private readonly TimeSpan _initialOffset;
    private readonly double _driftPpm;
    private readonly Func<DateTime> _utcNow;
    private long _correctionTicks;

    public SimulatedClock(TimeSpan initialOffset, double driftPpm = 0, DateTime? scenarioStartUtc = null,
        TimeSpan? baseTime = null, Func<DateTime>? utcNow = null)
    {
        _utcNow = utcNow ?? (() => DateTime.UtcNow);
        _initialOffset = initialOffset;
        _driftPpm = driftPpm;
        if (scenarioStartUtc is { } start)
        {
            _startUtc = start;
            _shownAtStart = BaseDay + (baseTime ?? new TimeSpan(10, 0, 0));
        }
        else
        {
            // Without a scenario, the clock shows real UTC time (+ offset).
            _startUtc = _utcNow().Date;
            _shownAtStart = _startUtc;
        }
    }

    /// <summary>Reads CLOCK_OFFSET_SECONDS, CLOCK_DRIFT_PPM, SCENARIO_START_UTC and SCENARIO_BASE_TIME.</summa
ry>
    public static SimulatedClock FromEnvironment(string offsetVariable = "CLOCK_OFFSET_SECONDS",
        string driftVariable = "CLOCK_DRIFT_PPM")
    {
        var offset = TimeSpan.FromSeconds(ReadDouble(offsetVariable));
        var drift = ReadDouble(driftVariable);
        DateTime? start = null;
        var startText = Environment.GetEnvironmentVariable("SCENARIO_START_UTC");
        if (!string.IsNullOrWhiteSpace(startText))
        {
            start = DateTime.Parse(startText, CultureInfo.InvariantCulture,
                DateTimeStyles.AdjustToUniversal | DateTimeStyles.AssumeUniversal);
        }
        var baseText = Environment.GetEnvironmentVariable("SCENARIO_BASE_TIME");
        TimeSpan? baseTime = string.IsNullOrWhiteSpace(baseText)
            ? null
            : TimeSpan.Parse(baseText, CultureInfo.InvariantCulture);
        return new SimulatedClock(offset, drift, start, baseTime);
    }
```

```
    private static double ReadDouble(string variable)
    {
        var text = Environment.GetEnvironmentVariable(variable);
        return string.IsNullOrWhiteSpace(text) ? 0 : double.Parse(text, CultureInfo.InvariantCulture);
    }

    public TimeSpan Correction => TimeSpan.FromTicks(Interlocked.Read(ref _correctionTicks));

    public double DriftPpm => _driftPpm;

    public DateTime Now
    {
        get
        {
            var elapsed = _utcNow() - _startUtc;
            var drift = TimeSpan.FromTicks((long)(elapsed.Ticks * _driftPpm / 1_000_000));
            return _shownAtStart + elapsed + _initialOffset + drift + Correction;
        }
    }

    /// <summary>Applies a correction to the application clock (the OS clock is not modified).</summary>
    public void Adjust(TimeSpan delta) => Interlocked.Add(ref _correctionTicks, delta.Ticks);
}
```

src/Clocks/Cristian.cs

```
namespace Clocks;

/// <summary>One request to the time server: local send/receive instants, server time and RTT.</summary>
public sealed record CristianSample(DateTime LocalSend, DateTime LocalReceive, DateTime ServerTime, TimeSpan Rt
t)
{
    /// <summary>Cristian: the server read its clock, on average, RTT/2 before the reply arrived.</summary>
    public DateTime EstimatedNow => ServerTime + Rtt / 2;

    /// <summary>Correction to add to the local clock.</summary>
    public TimeSpan Offset => EstimatedNow - LocalReceive;

    /// <summary>Worst-case error of the estimate: the server time was read somewhere inside the RTT.</summary>
    public TimeSpan Accuracy => Rtt / 2;
}

public static class Cristian
{
    /// <summary>
    /// Picks the sample with the lowest RTT among those below <paramref name="maxRtt"/>
    /// (the most precise one). Returns null when every sample was discarded.
    /// </summary>
    public static CristianSample? BestSample(IEnumerable<CristianSample> samples, TimeSpan maxRtt) =>
        samples.Where(s => s.Rtt <= maxRtt).OrderBy(s => s.Rtt).FirstOrDefault();
}
```

src/Clocks/Berkeley.cs

```
namespace Clocks;

/// <summary>Difference between a participant's clock and the coordinator's clock (null = did not answer).</sum
mary>
public sealed record BerkeleyReading(string Name, TimeSpan? Difference);

public sealed record BerkeleyAdjustment(string Name, TimeSpan? Difference, bool Included, TimeSpan? Adjustmen
t);

public sealed record BerkeleyRound(TimeSpan Average, IReadOnlyList<BerkeleyAdjustment> Adjustments);

public static class Berkeley
```

```
{
    /// <summary>
    /// Computes the average of the differences (the coordinator's own reading must be included with
    /// difference 0) and the adjustment each participant must apply: average - difference.
    /// With <paramref name="outlierThreshold"/> > 0 it uses a fault-tolerant average: readings that
    /// differ from the median by more than the threshold are left out of the average, but they are
    /// still told how to adjust. Participants that did not answer get no adjustment.
    /// </summary>
    public static BerkeleyRound Compute(IReadOnlyList<BerkeleyReading> readings, TimeSpan outlierThreshold)
    {
        var answered = readings.Where(r => r.Difference.HasValue).ToList();
        if (answered.Count == 0)
        {
            throw new ArgumentException("At least the coordinator reading is required", nameof(readings));
        }

        var sorted = answered.Select(r => r.Difference!.Value.Ticks).Order().ToList();
        var median = sorted.Count % 2 == 1
            ? sorted[sorted.Count / 2]
            : (sorted[sorted.Count / 2 - 1] + sorted[sorted.Count / 2]) / 2;

        bool IsIncluded(BerkeleyReading r) =>
            r.Difference.HasValue &&
            (outlierThreshold <= TimeSpan.Zero || Math.Abs(r.Difference.Value.Ticks - median) <= outlierThresho
ld.Ticks);

        var included = readings.Where(IsIncluded).ToList();
        var average = TimeSpan.FromTicks((long)included.Average(r => r.Difference!.Value.Ticks));

        var adjustments = readings
            .Select(r => new BerkeleyAdjustment(r.Name, r.Difference, IsIncluded(r),
                r.Difference.HasValue ? average - r.Difference.Value : null))
            .ToList();
        return new BerkeleyRound(average, adjustments);
    }
}
```

src/Clocks/LogicalClocks.cs

```
using System.Globalization;

namespace Clocks;

/// <summary>Lamport logical clock (thread-safe).</summary>
public sealed class LamportClock
{
    private readonly object _lock = new();
    private long _value;

    public LamportClock(long initial = 0) => _value = initial;

    public long Value
    {
        get { lock (_lock) return _value; }
    }

    /// <summary>Internal event or send: C = C + 1. The returned value travels with the message.</summary>
    public long Tick()
    {
        lock (_lock) return ++_value;
    }

    /// <summary>Receive: C = max(C, received) + 1.</summary>
    public long Receive(long received)
    {
        lock (_lock)
        {
            _value = Math.Max(_value, received) + 1;
            return _value;
        }
```

```
    }
}

/// <summary>
/// Vector clock, used only to check which events are really concurrent (something Lamport
/// timestamps alone cannot tell).
/// </summary>
public sealed class VectorClock
{
    private readonly object _lock = new();
    private readonly string _self;
    private readonly SortedDictionary<string, long> _entries;

    public VectorClock(string self, IEnumerable<string> processes)
    {
        _self = self;
        _entries = new SortedDictionary<string, long>(processes.ToDictionary(p => p, _ => 0L), StringComparer.O
rdinal);
        _entries.TryAdd(self, 0);
    }

    public string Tick()
    {
        lock (_lock)
        {
            _entries[_self]++;
            return Format(_entries);
        }
    }

    public string Receive(string received)
    {
        lock (_lock)
        {
            foreach (var (process, value) in Parse(received))
            {
                _entries[process] = Math.Max(_entries.GetValueOrDefault(process), value);
            }
            _entries[_self]++;
            return Format(_entries);
        }
    }

    public static string Format(IEnumerable<KeyValuePair<string, long>> entries) =>
        string.Join(",", entries.Select(e => $"{e.Key}={e.Value.ToString(CultureInfo.InvariantCulture)}"));

    public static Dictionary<string, long> Parse(string text) =>
        text.Split(',', StringSplitOptions.RemoveEmptyEntries)
            .Select(part => part.Split('='))
            .ToDictionary(kv => kv[0], kv => long.Parse(kv[1], CultureInfo.InvariantCulture));

    /// <summary>a happened-before b: a &lt;= b component-wise and a != b.</summary>
    public static bool HappenedBefore(string a, string b)
    {
        var va = Parse(a);
        var vb = Parse(b);
        var keys = va.Keys.Union(vb.Keys).ToList();
        return keys.All(k => va.GetValueOrDefault(k) <= vb.GetValueOrDefault(k)) &&
               keys.Any(k => va.GetValueOrDefault(k) < vb.GetValueOrDefault(k));
    }

    public static bool Concurrent(string a, string b) => !HappenedBefore(a, b) && !HappenedBefore(b, a) && a !=
b;
}
```

src/Clocks/EventProcess.cs

```
namespace Clocks;

public sealed record LoggedEvent(string Process, string Event, DateTime Physical, long Lamport, string Vector);
```

```
/// <summary>
/// A process of the distributed system: keeps its physical clock, its Lamport clock and a vector
/// clock, and logs every event with the three values. Every operation is done under one lock so
/// the log order of a process always matches its clock order.
/// </summary>
public sealed class EventProcess
{
    public const string LamportHeader = "X-Lamport-Clock";
    public const string VectorHeader = "X-Vector-Clock";

    private readonly object _lock = new();
    private readonly List<LoggedEvent> _events = [];
    private readonly LamportClock _lamport = new();
    private readonly VectorClock _vector;

    public EventProcess(string name, string vectorKey, IEnumerable<string> vectorKeys, SimulatedClock physical)
    {
        Name = name;
        Physical = physical;
        _vector = new VectorClock(vectorKey, vectorKeys);
    }

    public string Name { get; }

    public SimulatedClock Physical { get; }

    public IReadOnlyList<LoggedEvent> Events
    {
        get { lock (_lock) return _events.ToList(); }
    }

    /// <summary>Rule 1: internal event, C = C + 1.</summary>
    public LoggedEvent Internal(string description)
    {
        lock (_lock) return Record(description, _lamport.Tick(), _vector.Tick());
    }

    /// <summary>Rule 2: before sending, C = C + 1; the message carries C (and the vector).</summary>
    public LoggedEvent Send(string description)
    {
        lock (_lock) return Record(description, _lamport.Tick(), _vector.Tick());
    }

    /// <summary>Rule 3: on receive, C = max(C, received) + 1.</summary>
    public LoggedEvent Receive(string description, long receivedLamport, string receivedVector)
    {
        lock (_lock)
        {
            var before = _lamport.Value;
            var lamport = _lamport.Receive(receivedLamport);
            var note = receivedLamport > before
                ? $" (recibido L={receivedLamport} > local L={before}: salta a {lamport})"
                : $" (recibido L={receivedLamport} <= local L={before}: local + 1)";
            return Record(description + note, lamport, _vector.Receive(receivedVector));
        }
    }

    private LoggedEvent Record(string description, long lamport, string vector)
    {
        var logged = new LoggedEvent(Name, description, Physical.Now, lamport, vector);
        _events.Add(logged);
        Console.WriteLine($"[{Name}] fisico {TimeFormat.Time(logged.Physical)} | Lamport {lamport,3} | vector
[{vector}] | {description}");
        return logged;
    }
}
```

src/Cristian.TimeServer/Program.cs

```
// Exercise 1 - Time server for Cristian's algorithm.
//
// GET /api/time                    -> current time of the server clock
// GET /api/time?requestDelayMs=X&responseDelayMs=Y
//      simulates network latency: waits X ms before reading the clock (request travelling
//      to the server) and Y ms after reading it (response travelling back).
using Clocks;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var name = Environment.GetEnvironmentVariable("SERVER_NAME") ?? "time-server";
var clock = SimulatedClock.FromEnvironment();

app.MapGet("/api/time", async (int? requestDelayMs, int? responseDelayMs, string? client) =>
{
    if (requestDelayMs > 0) await Task.Delay(requestDelayMs.Value);
    var now = clock.Now;
    Console.WriteLine($"[{name}] hora {TimeFormat.Time(now)} -> {client ?? "?"}" +
                      (requestDelayMs > 0 || responseDelayMs > 0
                          ? $" (latencia simulada: ida {requestDelayMs ?? 0} ms, vuelta {responseDelayMs ?? 0}
ms)"
                          : ""));
    if (responseDelayMs > 0) await Task.Delay(responseDelayMs.Value);
    return Results.Ok(new { Server = name, ServerTime = now });
});

app.MapGet("/health", () => Results.Ok(new { Status = "UP" }));

Console.WriteLine($"[{name}] servidor de tiempo iniciado, hora local {TimeFormat.Time(clock.Now)}");
app.Run();
```

src/Cristian.Client/Program.cs

```
// Exercise 1 - Client of Cristian's algorithm.
//
// For each synchronization it:
//   1. asks the time server for its time (REST),
//   2. records the local send instant (T0) and 3. the local receive instant (T1),
//   4. computes the round trip time (RTT, measured with a monotonic Stopwatch),
//   5. estimates the current time as serverTime + RTT/2,
//   6. adjusts its application clock (never the OS clock) and
//   7. prints the time before and after the adjustment.
// It takes several samples and keeps the one with the lowest RTT (samples above MAX_RTT_MS
// are discarded).
//
// Environment:
//   CLIENT_NAME, TIME_SERVER_URL, CLOCK_OFFSET_SECONDS, CLOCK_DRIFT_PPM
//   PHASES               normal,simetrica,asimetrica,alta (simulated network conditions)
//   SAMPLES              samples per synchronization (default 3)
//   MAX_RTT_MS           samples with a higher RTT are discarded (default 1000)
//   ROUNDS, SYNC_INTERVAL_SECONDS  periodic resynchronization after the phases
//   EVAL_SERVER_OFFSET_SECONDS, EVAL_SERVER_DRIFT_PPM
//       ONLY used to print the real error: the algorithm never reads them.
using System.Diagnostics;
using System.Globalization;
using System.Net.Http.Json;
using Clocks;

var name = Env("CLIENT_NAME", "cliente");
var serverUrl = Env("TIME_SERVER_URL", "http://time-server:8080");
var samplesPerSync = int.Parse(Env("SAMPLES", "3"), CultureInfo.InvariantCulture);
var maxRtt = TimeSpan.FromMilliseconds(double.Parse(Env("MAX_RTT_MS", "1000"), CultureInfo.InvariantCulture));
var rounds = int.Parse(Env("ROUNDS", "0"), CultureInfo.InvariantCulture);
var interval = TimeSpan.FromSeconds(double.Parse(Env("SYNC_INTERVAL_SECONDS", "5"), CultureInfo.InvariantCultur
e));
```

```
var phases = Env("PHASES", "normal").Split(',', StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.Trim
Entries);

var clock = SimulatedClock.FromEnvironment();
// Reference used only to measure how good the estimate was.
var serverTruth = SimulatedClock.FromEnvironment("EVAL_SERVER_OFFSET_SECONDS", "EVAL_SERVER_DRIFT_PPM");

using var http = new HttpClient { BaseAddress = new Uri(serverUrl), Timeout = TimeSpan.FromSeconds(4) };

var networks = new Dictionary<string, (int Request, int Response, string Description)>
{
    ["normal"] = (0, 0, "red Docker sin latencia agregada"),
    ["simetrica"] = (200, 200, "latencia simetrica: 200 ms de ida y 200 ms de vuelta"),
    ["asimetrica"] = (50, 450, "latencia asimetrica: 50 ms de ida y 450 ms de vuelta"),
    ["alta"] = (1500, 1500, $"RTT muy alto: 1500 ms de ida y 1500 ms de vuelta (limite {maxRtt.TotalMillisecond
s} ms)"),
};

Log($"iniciado. Hora local {TimeFormat.Time(clock.Now)} | servidor {serverUrl} | deriva {clock.DriftPpm} ppm");
Log($"diferencia real con el servidor antes de sincronizar: {TimeFormat.Delta(clock.Now - serverTruth.Now)}");
await WaitForServer();

foreach (var phase in phases)
{
    var network = networks[phase];
    Log($"=== Fase '{phase}': {network.Description} ===");
    await Synchronize(network.Request, network.Response);
}

for (var round = 1; round <= rounds; round++)
{
    await Task.Delay(interval);
    Log($"=== Resincronizacion periodica {round}/{rounds} ===");
    await Synchronize(0, 0);
}

Log("fin");
return;

async Task Synchronize(int requestDelayMs, int responseDelayMs)
{
    var samples = new List<CristianSample>();
    for (var i = 1; i <= samplesPerSync; i++)
    {
        var t0 = clock.Now;                      // 2. local send instant
        var stopwatch = Stopwatch.StartNew();
        try
        {
            var reply = await http.GetFromJsonAsync<TimeReply>(
                $"/api/time?client={name}&requestDelayMs={requestDelayMs}&responseDelayMs={responseDelayMs}");
            stopwatch.Stop();
            var t1 = clock.Now;                  // 3. local receive instant
            var sample = new CristianSample(t0, t1, reply!.ServerTime, stopwatch.Elapsed);   // 4. RTT
            samples.Add(sample);
            Log($"  muestra {i}: T0={TimeFormat.Time(t0)} T1={TimeFormat.Time(t1)} " +
                $"servidor={TimeFormat.Time(sample.ServerTime)} RTT={TimeFormat.Ms(sample.Rtt)}" +
                (sample.Rtt > maxRtt ? "  -> DESCARTADA (RTT mayor al limite)" : ""));
        }
        catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
        {
            Log($"  muestra {i}: el servidor de tiempo NO responde ({ex.GetType().Name}: {ex.Message.Split
('\n')[0]})");
        }
    }

    var best = Cristian.BestSample(samples, maxRtt);
    if (best is null)
    {
        Log($"  sin muestras validas: se mantiene el reloj (correccion acumulada {TimeFormat.Delta(clock.Correc
tion)}). " +
            $"Error real actual: {TimeFormat.SignedMs(clock.Now - serverTruth.Now)}");
        return;
    }
```

```
    var before = clock.Now;
    clock.Adjust(best.Offset);                  // 6. adjust the application clock
    var after = clock.Now;
    Log($"  mejor muestra RTT={TimeFormat.Ms(best.Rtt)} -> hora estimada = servidor + RTT/2 (precision +/-{Time
Format.Ms(best.Accuracy)})");
    Log($"  antes {TimeFormat.Time(before)} | despues {TimeFormat.Time(after)} | ajuste {TimeFormat.Delta(best.
Offset)} | " +
        $"error real {TimeFormat.SignedMs(after - serverTruth.Now)}");
}

async Task WaitForServer()
{
    for (var i = 0; i < 30; i++)
    {
        try
        {
            using var response = await http.GetAsync("/health");
            if (response.IsSuccessStatusCode) return;
        }
        catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
        {
        }
        await Task.Delay(500);
    }
}

void Log(string message) => Console.WriteLine($"[{name} {TimeFormat.Time(clock.Now)}] {message}");

static string Env(string variable, string fallback) =>
    Environment.GetEnvironmentVariable(variable) is { Length: > 0 } value ? value : fallback;

internal sealed record TimeReply(string Server, DateTime ServerTime);
```

src/Berkeley.Node/Program.cs

```
// Exercise 2 - Participant node of Berkeley's algorithm.
//
// GET  /api/clock          -> current time of the node's simulated clock
// POST /api/clock/adjust   -> { "adjustmentMs": 1234.5 } applies the adjustment sent by the coordinator
using Clocks;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var name = Environment.GetEnvironmentVariable("NODE_NAME") ?? "nodo";
var clock = SimulatedClock.FromEnvironment();

app.MapGet("/api/clock", () => Results.Ok(new ClockReply(name, clock.Now)));

app.MapPost("/api/clock/adjust", (AdjustRequest request) =>
{
    var adjustment = TimeSpan.FromMilliseconds(request.AdjustmentMs);
    var before = clock.Now;
    clock.Adjust(adjustment);
    var after = clock.Now;
    Console.WriteLine($"[{name}] ajuste recibido {TimeFormat.Delta(adjustment)}: " +
                      $"{TimeFormat.Time(before)} -> {TimeFormat.Time(after)}");
    return Results.Ok(new ClockReply(name, after));
});

app.MapGet("/health", () => Results.Ok(new { Status = "UP" }));

Console.WriteLine($"[{name}] nodo iniciado, hora local {TimeFormat.Time(clock.Now)} (deriva {clock.DriftPpm} pp
m)");
app.Run();

internal sealed record ClockReply(string Name, DateTime Time);
```

```
internal sealed record AdjustRequest(double AdjustmentMs);
```

src/Berkeley.Coordinator/Program.cs

```
// Exercise 2 - Coordinator of Berkeley's algorithm.
//
// Every INTERVAL_SECONDS it:
//   1. asks every node for its current time,
//   2. computes each difference with its own clock (compensating RTT/2),
//   3. averages the differences including its own (0),
//   4. computes the adjustment of each server (average - difference),
//   5. sends each node its adjustment and applies its own,
//   6. repeats after the configured interval.
//
// Environment:
//   COORDINATOR_NAME, NODES (comma separated URLs), CLOCK_OFFSET_SECONDS, CLOCK_DRIFT_PPM
//   INTERVAL_SECONDS (default 5), ROUNDS (0 = forever)
//   OUTLIER_THRESHOLD_SECONDS  0 = plain average; > 0 = fault-tolerant average that leaves out
//                              clocks that differ from the median by more than the threshold
using System.Diagnostics;
using System.Globalization;
using System.Net.Http.Json;
using Clocks;

var name = Env("COORDINATOR_NAME", "coordinador");
var nodes = Env("NODES", "").Split(',', StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntrie
s);
var interval = TimeSpan.FromSeconds(double.Parse(Env("INTERVAL_SECONDS", "5"), CultureInfo.InvariantCulture));
var rounds = int.Parse(Env("ROUNDS", "0"), CultureInfo.InvariantCulture);
var threshold = TimeSpan.FromSeconds(double.Parse(Env("OUTLIER_THRESHOLD_SECONDS", "0"), CultureInfo.InvariantC
ulture));
var clock = SimulatedClock.FromEnvironment();

using var http = new HttpClient { Timeout = TimeSpan.FromSeconds(2) };

Log($"iniciado. Hora local {TimeFormat.Time(clock.Now)} | nodos: {string.Join(", ", nodes)} | intervalo {interv
al.TotalSeconds} s | " +
    (threshold > TimeSpan.Zero
        ? $"promedio tolerante a fallas (descarta relojes a mas de {threshold.TotalSeconds} s de la mediana)"
        : "promedio simple"));
await Task.Delay(TimeSpan.FromSeconds(2)); // let the nodes start

for (var round = 1; rounds == 0 || round <= rounds; round++)
{
    await RunRound(round);
    if (rounds == 0 || round < rounds) await Task.Delay(interval);
}

Log("fin");
return;

async Task RunRound(int round)
{
    Log($"===== Ronda {round} =====");

    // 1-2. Poll every node in parallel and compute its difference with the coordinator.
    var polls = await Task.WhenAll(nodes.Select(Poll));
    var readings = new List<BerkeleyReading> { new(name, TimeSpan.Zero) };
    readings.AddRange(polls.Select(p => new BerkeleyReading(p.Name, p.Difference)));
    var coordinatorNow = clock.Now;

    // 3-4. Average and adjustments.
    var result = Berkeley.Compute(readings, threshold);

    // Every clock is shown at the same instant (coordinator time + difference), so the columns compare.
    Log($"{"participante",-14}{"hora",-15}{"diferencia",-13}{"en promedio",-13}{"ajuste",-13}hora ajustada");
    foreach (var adjustment in result.Adjustments)
    {
        if (adjustment.Difference is null)
```

```
        {
            var error = polls.FirstOrDefault(p => p.Name == adjustment.Name)?.Error;
            Log($"{adjustment.Name,-14}{"SIN RESPUESTA",-15}{"-",-13}{"no",-13}{"-",-13}-  ({error})");
            continue;
        }
        var read = coordinatorNow + adjustment.Difference.Value;
        Log($"{adjustment.Name,-14}{TimeFormat.Time(read),-15}{TimeFormat.Delta(adjustment.Difference.Value),-1
3}" +
            $"{(adjustment.Included ? "si" : "NO (atipico)"),-13}{TimeFormat.Delta(adjustment.Adjustment!.Valu
e),-13}" +
            $"{TimeFormat.Time(read + adjustment.Adjustment.Value)}");
    }

    var answered = readings.Where(r => r.Difference.HasValue).Select(r => r.Difference!.Value).ToList();
    Log($"promedio de diferencias: {TimeFormat.Delta(result.Average)} | " +
        $"dispersion antes del ajuste (max - min): {TimeFormat.Delta(answered.Max() - answered.Min())}");

    // 5. Send each node its adjustment; apply the coordinator's own.
    foreach (var adjustment in result.Adjustments.Where(a => a.Adjustment.HasValue))
    {
        if (adjustment.Name == name)
        {
            clock.Adjust(adjustment.Adjustment!.Value);
            continue;
        }
        var url = polls.First(p => p.Name == adjustment.Name).Url;
        try
        {
            using var response = await http.PostAsJsonAsync($"{url}/api/clock/adjust",
                new { AdjustmentMs = adjustment.Adjustment!.Value.TotalMilliseconds });
            response.EnsureSuccessStatusCode();
        }
        catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
        {
            Log($"no se pudo enviar el ajuste a {adjustment.Name}: {ex.Message}");
        }
    }
}

async Task<Poll> Poll(string url)
{
    var host = new Uri(url).Host;
    var stopwatch = Stopwatch.StartNew();
    try
    {
        var reply = await http.GetFromJsonAsync<ClockReply>($"{url}/api/clock");
        stopwatch.Stop();
        // Same idea as Cristian: the node read its clock about RTT/2 before the reply arrived.
        var estimated = reply!.Time + stopwatch.Elapsed / 2;
        return new Poll(host, url, estimated - clock.Now, null);
    }
    catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
    {
        return new Poll(host, url, null, ex is TaskCanceledException ? "timeout" : ex.Message.Split('(')[0].Tri
m());
    }
}

void Log(string message) => Console.WriteLine($"[{name} {TimeFormat.Time(clock.Now)}] {message}");

static string Env(string variable, string fallback) =>
    Environment.GetEnvironmentVariable(variable) is { Length: > 0 } value ? value : fallback;

internal sealed record ClockReply(string Name, DateTime Time);

internal sealed record Poll(string Name, string Url, TimeSpan? Difference, string? Error);
```

src/Lamport.Orders/Program.cs

```
// Exercise 3 - OrdersService: receives orders and coordinates inventory and payment.
//
// POST /api/orders    { productId, quantity, amount }  runs the 9-step sequence of the case study
// GET  /api/events    -> event log of this service
// GET  /api/timeline  -> events of the three services merged and ordered by Lamport and by
//                        physical time (text report). Reading the logs is an observation, not
//                        part of the computation, so it does not touch the clocks.
using System.Net.Http.Json;
using Clocks;
using Lamport.Orders;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddHttpClient("inventory", c => c.BaseAddress = new Uri(Env("INVENTORY_URL", "http://inventory
-service:8080")));
builder.Services.AddHttpClient("payments", c => c.BaseAddress = new Uri(Env("PAYMENTS_URL", "http://payments-se
rvice:8080")));
var app = builder.Build();

var process = new EventProcess("OrdersService", "O", ["I", "O", "P"], SimulatedClock.FromEnvironment());
var nextOrder = 0;

app.MapPost("/api/orders", async (OrderRequest request, IHttpClientFactory factory) =>
{
    var orderId = $"ORD-{Interlocked.Increment(ref nextOrder):000}";

    // 1. OrdersService recibe un pedido
    process.Internal($"Pedido recibido {orderId}: {request.Quantity} x {request.ProductId}");

    // 2. OrdersService envia una solicitud a InventoryService
    var reserve = await Exchange(factory.CreateClient("inventory"), "/api/inventory/reserve",
        new { OrderId = orderId, request.ProductId, request.Quantity },
        $"Mensaje enviado a InventoryService (pedido {orderId})",
        $"Respuesta de InventoryService recibida (pedido {orderId})");
    var reservation = await reserve.ReadFromJsonAsync<ReserveReply>();

    if (reservation is not { Reserved: true })
    {
        process.Internal($"Pedido {orderId} rechazado: sin stock");
        return Results.Conflict(new { OrderId = orderId, Status = "REJECTED" });
    }

    // 6. OrdersService solicita el procesamiento del pago
    var payment = await Exchange(factory.CreateClient("payments"), "/api/payments",
        new { OrderId = orderId, request.Amount },
        $"Solicitud de pago enviada a PaymentsService (pedido {orderId})",
        $"Confirmacion de pago recibida (pedido {orderId})");
    await payment.ReadFromJsonAsync<object>();

    // 9. OrdersService confirma el pedido
    var confirmed = process.Internal($"Pedido {orderId} confirmado");
    return Results.Ok(new { OrderId = orderId, Status = "CONFIRMED", confirmed.Lamport });
});

app.MapGet("/api/events", () => Results.Ok(process.Events));

app.MapGet("/api/timeline", async (IHttpClientFactory factory) =>
{
    var events = new List<LoggedEvent>(process.Events);
    events.AddRange(await factory.CreateClient("inventory").GetFromJsonAsync<List<LoggedEvent>>("/api/events")
?? []);
    events.AddRange(await factory.CreateClient("payments").GetFromJsonAsync<List<LoggedEvent>>("/api/events")
?? []);
    return Results.Text(Timeline.Render(events));
});

app.MapGet("/health", () => Results.Ok(new { Status = "UP" }));

Console.WriteLine($"[{process.Name}] iniciado, reloj fisico {TimeFormat.Time(process.Physical.Now)}");
app.Run();
```

```
return;

// Sends a message with the clocks in the headers and records the reply as a receive event.
async Task<HttpContent> Exchange(HttpClient client, string path, object body, string sendEvent, string receiveE
vent)
{
    var sent = process.Send(sendEvent);
    using var message = new HttpRequestMessage(HttpMethod.Post, path) { Content = JsonContent.Create(body) };
    message.Headers.Add(EventProcess.LamportHeader, sent.Lamport.ToString());
    message.Headers.Add(EventProcess.VectorHeader, sent.Vector);
    var response = await client.SendAsync(message);
    process.Receive(receiveEvent,
        long.Parse(response.Headers.GetValues(EventProcess.LamportHeader).First()),
        response.Headers.GetValues(EventProcess.VectorHeader).First());
    return response.Content;
}

static string Env(string variable, string fallback) =>
    Environment.GetEnvironmentVariable(variable) is { Length: > 0 } value ? value : fallback;

internal sealed record OrderRequest(string ProductId, int Quantity, decimal Amount);

internal sealed record ReserveReply(string OrderId, bool Reserved, int Available);
```

src/Lamport.Orders/Timeline.cs

```
using System.Text;
using Clocks;

namespace Lamport.Orders;

/// <summary>Builds the merged report of the three services.</summary>
public static class Timeline
{
    public static string Render(IReadOnlyList<LoggedEvent> events)
    {
        var text = new StringBuilder();

        // Lamport gives a partial order; the process name breaks ties to get a total order.
        var byLamport = events.OrderBy(e => e.Lamport).ThenBy(e => e.Process, StringComparer.Ordinal).ToList();
        text.AppendLine("=== Orden total por reloj de Lamport (desempate por nombre de proceso) ===");
        AppendTable(text, byLamport);

        var byPhysical = events.OrderBy(e => e.Physical).ToList();
        text.AppendLine();
        text.AppendLine("=== Orden por reloj fisico (hora local de cada servicio) ===");
        AppendTable(text, byPhysical);

        // a -> b (by vector clocks) but b has an earlier physical timestamp than a.
        var violations = (from a in events
                          from b in events
                          where VectorClock.HappenedBefore(a.Vector, b.Vector) && b.Physical < a.Physical
                          select (a, b)).ToList();
        text.AppendLine();
        text.AppendLine($"=== Inconsistencias del reloj fisico: {violations.Count} pares donde a -> b pero hora
(b) < hora(a) ===");
        // Show each message as (send -> receive): b is the first event of its process caused by a,
        // and a is the latest event of the other process that b depends on.
        var closest = violations
            .Where(v => v.a.Process != v.b.Process)
            .GroupBy(v => v.b)
            .Select(g => g.MaxBy(v => v.a.Lamport))
            .Where(v => !events.Any(e => e.Process == v.b.Process && e.Lamport < v.b.Lamport &&
                                         VectorClock.HappenedBefore(v.a.Vector, e.Vector)))
            .OrderBy(v => v.b.Lamport)
            .Take(6);
        foreach (var (a, b) in closest)
        {
            text.AppendLine($"  {Describe(a)}  ->  {Describe(b)}  (hora(b) - hora(a) = {TimeFormat.Delta(b.Phys
ical - a.Physical)})");
```

```
        }

        // Lamport says L(a) < L(b), but the events are not causally related.
        var concurrent = (from a in events
                          from b in events
                          where a.Lamport < b.Lamport && VectorClock.Concurrent(a.Vector, b.Vector)
                          select (a, b)).ToList();
        text.AppendLine();
        text.AppendLine($"=== Eventos concurrentes con L(a) < L(b): {concurrent.Count} pares (Lamport no los di
stingue) ===");
        foreach (var (a, b) in concurrent.Where(v => v.a.Process != v.b.Process).Take(4))
        {
            text.AppendLine($"  {Describe(a)}  ||  {Describe(b)}");
        }

        return text.ToString();
    }

    private static string Describe(LoggedEvent e) => $"{Short(e.Process)} L={e.Lamport} [{e.Vector}] \"{Cut(e.E
vent, 38)}\"";

    private static void AppendTable(StringBuilder text, IEnumerable<LoggedEvent> events)
    {
        text.AppendLine($"{"proceso",-18}{"reloj fisico",-15}{"Lamport",-9}{"vector",-16}evento");
        foreach (var e in events)
        {
            text.AppendLine($"{e.Process,-18}{TimeFormat.Time(e.Physical),-15}{e.Lamport,-9}{e.Vector,-16}{Cut
(e.Event, 95)}");
        }
    }

    private static string Short(string process) => process.Replace("Service", "");

    private static string Cut(string value, int max) => value.Length <= max ? value : value[..(max - 3)] +
"...";
}
```

src/Lamport.Inventory/Program.cs

```
// Exercise 3 - InventoryService: verifies and reserves products.
//
// POST /api/inventory/reserve  { orderId, productId, quantity }
//      headers X-Lamport-Clock / X-Vector-Clock with the sender's clocks;
//      the response carries this service's clocks in the same headers.
// GET  /api/events             -> event log (physical time, Lamport, vector)
using System.Collections.Concurrent;
using Clocks;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var process = new EventProcess("InventoryService", "I", ["I", "O", "P"], SimulatedClock.FromEnvironment());
var stock = new ConcurrentDictionary<string, int>(StringComparer.OrdinalIgnoreCase)
{
    ["P001"] = 10,
    ["P002"] = 3,
    ["P003"] = 0,
};

app.MapPost("/api/inventory/reserve", (ReserveRequest request, HttpRequest http, HttpResponse response) =>
{
    // 3. InventoryService registra la recepcion
    process.Receive($"Mensaje recibido de OrdersService (pedido {request.OrderId})",
        long.Parse(http.Headers[EventProcess.LamportHeader]!), http.Headers[EventProcess.VectorHeader]!);

    // 4. InventoryService verifica el inventario
    var available = stock.GetValueOrDefault(request.ProductId);
    var reserved = available >= request.Quantity &&
                   stock.TryUpdate(request.ProductId, available - request.Quantity, available);
    process.Internal($"Inventario verificado: {request.ProductId} stock {available}, pedido {request.Quantity}
```

```
-> " +
                    (reserved ? "reservado" : "sin stock"));

    // 5. InventoryService envia una respuesta
    var sent = process.Send($"Respuesta enviada a OrdersService (pedido {request.OrderId})");
    response.Headers[EventProcess.LamportHeader] = sent.Lamport.ToString();
    response.Headers[EventProcess.VectorHeader] = sent.Vector;
    return Results.Ok(new { request.OrderId, Reserved = reserved, Available = available });
});

app.MapGet("/api/events", () => Results.Ok(process.Events));
app.MapGet("/health", () => Results.Ok(new { Status = "UP" }));

Console.WriteLine($"[{process.Name}] iniciado, reloj fisico {TimeFormat.Time(process.Physical.Now)}");
app.Run();

internal sealed record ReserveRequest(string OrderId, string ProductId, int Quantity);
```

src/Lamport.Payments/Program.cs

```
// Exercise 3 - PaymentsService: processes payments.
//
// POST /api/payments  { orderId, amount }  (with X-Lamport-Clock / X-Vector-Clock headers)
// GET  /api/events    -> event log
//
// PRELOAD_EVENTS: internal events executed at startup (payments of other, unrelated orders),
// so this process starts with a Lamport clock ahead of the others.
using System.Globalization;
using Clocks;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var process = new EventProcess("PaymentsService", "P", ["I", "O", "P"], SimulatedClock.FromEnvironment());
var preload = int.Parse(Environment.GetEnvironmentVariable("PRELOAD_EVENTS") ?? "0", CultureInfo.InvariantCultu
re);
for (var i = 1; i <= preload; i++)
{
    process.Internal($"Pago de otro pedido procesado (#{i}, sin relacion con este flujo)");
}

app.MapPost("/api/payments", async (PaymentRequest request, HttpRequest http, HttpResponse response) =>
{
    // 7. PaymentsService registra y procesa la solicitud
    process.Receive($"Solicitud de pago recibida (pedido {request.OrderId})",
        long.Parse(http.Headers[EventProcess.LamportHeader]!), http.Headers[EventProcess.VectorHeader]!);
    await Task.Delay(50); // talking to the payment gateway
    process.Internal($"Pago procesado: ${request.Amount.ToString("0.00", CultureInfo.InvariantCulture)} aprobad
o");

    // 8. PaymentsService envia una confirmacion
    var sent = process.Send($"Confirmacion enviada a OrdersService (pedido {request.OrderId})");
    response.Headers[EventProcess.LamportHeader] = sent.Lamport.ToString();
    response.Headers[EventProcess.VectorHeader] = sent.Vector;
    return Results.Ok(new { request.OrderId, Approved = true });
});

app.MapGet("/api/events", () => Results.Ok(process.Events));
app.MapGet("/health", () => Results.Ok(new { Status = "UP" }));

Console.WriteLine($"[{process.Name}] iniciado, reloj fisico {TimeFormat.Time(process.Physical.Now)}");
app.Run();

internal sealed record PaymentRequest(string OrderId, decimal Amount);
```

tests/Clocks.Tests/ClockTests.cs

```
using Clocks;
using Xunit;

namespace Clocks.Tests;

public class SimulatedClockTests
{
    private static readonly DateTime Start = new(2026, 11, 11, 20, 0, 0, DateTimeKind.Utc);

    [Fact]
    public void Shows_the_case_study_times_at_the_scenario_start()
    {
        var real = Start;
        var serverB = new SimulatedClock(TimeSpan.FromSeconds(8), 0, Start, null, () => real);
        var serverC = new SimulatedClock(TimeSpan.FromSeconds(-5), 0, Start, null, () => real);

        Assert.Equal("10:00:08.000", TimeFormat.Time(serverB.Now));
        Assert.Equal("09:59:55.000", TimeFormat.Time(serverC.Now));
    }

    [Fact]
    public void Drift_accumulates_and_adjust_changes_only_the_application_clock()
    {
        var real = Start;
        var clock = new SimulatedClock(TimeSpan.Zero, driftPpm: 1000, Start, null, () => real);

        real = Start.AddSeconds(10); // 1000 ppm = 1 ms per second
        Assert.Equal("10:00:10.010", TimeFormat.Time(clock.Now));

        clock.Adjust(TimeSpan.FromMilliseconds(-10));
        Assert.Equal("10:00:10.000", TimeFormat.Time(clock.Now));
        Assert.Equal(TimeSpan.FromMilliseconds(-10), clock.Correction);
    }
}

public class CristianTests
{
    private static readonly DateTime T = new(2026, 1, 1, 10, 0, 0, DateTimeKind.Utc);

    [Fact]
    public void Estimates_server_time_plus_half_the_round_trip()
    {
        // Client A (10:00:00) asks the server (10:00:02); RTT 100 ms.
        var sample = new CristianSample(T, T.AddMilliseconds(100), T.AddSeconds(2).AddMilliseconds(50),
            TimeSpan.FromMilliseconds(100));

        Assert.Equal(T.AddSeconds(2).AddMilliseconds(100), sample.EstimatedNow);
        Assert.Equal(TimeSpan.FromSeconds(2), sample.Offset);
        Assert.Equal(TimeSpan.FromMilliseconds(50), sample.Accuracy);
    }

    [Fact]
    public void Best_sample_is_the_lowest_rtt_and_slow_samples_are_discarded()
    {
        CristianSample S(int rttMs) => new(T, T, T, TimeSpan.FromMilliseconds(rttMs));
        var samples = new[] { S(40), S(12), S(3000), S(25) };

        Assert.Equal(TimeSpan.FromMilliseconds(12), Cristian.BestSample(samples, TimeSpan.FromSeconds(1))!.Rt
t);
        Assert.Null(Cristian.BestSample(new[] { S(3000), S(2500) }, TimeSpan.FromSeconds(1)));
    }
}

public class BerkeleyTests
{
    private static BerkeleyReading R(string name, double? seconds) =>
        new(name, seconds is null ? null : TimeSpan.FromSeconds(seconds.Value));

    [Fact]
```

```
    public void Everyone_moves_to_the_average_including_the_coordinator()
    {
        var round = Berkeley.Compute(new[] { R("coord", 0), R("n1", 10), R("n2", -6), R("n3", 4) }, TimeSpan.Ze
ro);

        Assert.Equal(TimeSpan.FromSeconds(2), round.Average);
        Assert.Equal(new double?[] { 2, -8, 8, -2 },
            round.Adjustments.Select(a => a.Adjustment?.TotalSeconds).ToArray());
    }

    [Fact]
    public void A_five_minute_outlier_drags_the_plain_average()
    {
        var round = Berkeley.Compute(new[] { R("coord", 0), R("n1", 5), R("n2", -3), R("n3", 300) }, TimeSpan.Z
ero);

        Assert.Equal(75.5, round.Average.TotalSeconds);
    }

    [Fact]
    public void Fault_tolerant_average_ignores_the_outlier_but_still_corrects_it()
    {
        var round = Berkeley.Compute(new[] { R("coord", 0), R("n1", 5), R("n2", -3), R("n3", 300) },
            TimeSpan.FromSeconds(30));

        Assert.Equal(2.0 / 3, round.Average.TotalSeconds, precision: 6);
        var outlier = round.Adjustments.Single(a => a.Name == "n3");
        Assert.False(outlier.Included);
        Assert.Equal(2.0 / 3 - 300, outlier.Adjustment!.Value.TotalSeconds, precision: 6);
    }

    [Fact]
    public void Disconnected_participants_are_left_out_and_get_no_adjustment()
    {
        var round = Berkeley.Compute(new[] { R("coord", 0), R("n1", 6), R("n2", null) }, TimeSpan.Zero);

        Assert.Equal(TimeSpan.FromSeconds(3), round.Average);
        Assert.Null(round.Adjustments.Single(a => a.Name == "n2").Adjustment);
    }
}

public class LogicalClockTests
{
    [Fact]
    public void Lamport_ticks_on_local_events_and_takes_the_max_on_receive()
    {
        var orders = new LamportClock();
        var inventory = new LamportClock();
        var payments = new LamportClock(initial: 10);

        Assert.Equal(1, orders.Tick());                // order received
        var sent = orders.Tick();                       // message to inventory carries 2
        Assert.Equal(3, inventory.Receive(sent));       // higher than its own 0 -> jumps to 3
        Assert.Equal(11, payments.Receive(sent));       // lower than its own 10 -> 11
    }

    [Fact]
    public void Vector_clocks_detect_concurrency_that_lamport_cannot()
    {
        var all = new[] { "I", "O", "P" };
        var orders = new VectorClock("O", all);
        var payments = new VectorClock("P", all);

        var a = orders.Tick();                          // O=1           Lamport 1
        payments.Tick();
        var b = payments.Tick();                        // P=2           Lamport 2
        var c = payments.Receive(orders.Tick());        // after O sends Lamport 3

        Assert.True(VectorClock.Concurrent(a, b));      // L(a)=1 < L(b)=2, yet concurrent
        Assert.True(VectorClock.HappenedBefore(a, c));
        Assert.True(VectorClock.HappenedBefore(b, c));
```

```
    }
}
```

Dockerfile

```
# One Dockerfile for every service: the project to build is passed with --build-arg PROJECT=<name>
# (docker-compose.yml does it for each service).
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
ARG PROJECT
WORKDIR /src
COPY src/Clocks/Clocks.csproj src/Clocks/
COPY src/${PROJECT}/${PROJECT}.csproj src/${PROJECT}/
RUN dotnet restore src/${PROJECT}/${PROJECT}.csproj
COPY src/Clocks/ src/Clocks/
COPY src/${PROJECT}/ src/${PROJECT}/
RUN dotnet publish src/${PROJECT}/${PROJECT}.csproj -c Release -o /app --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:8.0
ARG PROJECT
ENV APP_DLL=${PROJECT}.dll
WORKDIR /app
COPY --from=build /app .
ENTRYPOINT ["sh", "-c", "exec dotnet \"$APP_DLL\""]
```

docker-compose.yml

```
# One profile per exercise:
#   docker compose --profile cristian up --build
#   docker compose --profile berkeley up --build
#   docker compose --profile lamport  up --build
#
# Clock skew is simulated in each application (containers share the host clock):
#   CLOCK_OFFSET_SECONDS  initial offset of the clock
#   CLOCK_DRIFT_PPM       drift in parts per million (1000 ppm = 1 ms per second)
#   SCENARIO_START_UTC    real instant that is shown as 10:00:00 (set by the scripts; empty = real UTC)

x-common-env: &common-env
  SCENARIO_START_UTC: ${SCENARIO_START_UTC:-}
  Logging__LogLevel__Default: Warning

services:
  # ------------------------------------------------------------------ Exercise 1: Cristian
  time-server:
    profiles: [cristian]
    build: { context: ., args: { PROJECT: Cristian.TimeServer } }
    image: relojeria/cristian-time-server
    environment:
      <<: *common-env
      SERVER_NAME: time-server
      CLOCK_OFFSET_SECONDS: "2"            # 10:00:02
    networks: [relojeria]

  client-a:
    profiles: [cristian]
    build: { context: ., args: { PROJECT: Cristian.Client } }
    image: relojeria/cristian-client
    depends_on: [time-server]
    environment: &cristian-client
      <<: *common-env
      CLIENT_NAME: servidor-A
      TIME_SERVER_URL: http://time-server:8080
      CLOCK_OFFSET_SECONDS: "0"            # 10:00:00
      PHASES: normal,simetrica,asimetrica,alta
      ROUNDS: ${CRISTIAN_ROUNDS:-4}
      SYNC_INTERVAL_SECONDS: "4"
      EVAL_SERVER_OFFSET_SECONDS: "2"      # only to print the real error
    networks: [relojeria]
```

```
  client-b:
    profiles: [cristian]
    image: relojeria/cristian-client
    depends_on: [time-server, client-a]
    environment:
      <<: *cristian-client
      CLIENT_NAME: servidor-B
      CLOCK_OFFSET_SECONDS: "8"            # 10:00:08
      CLOCK_DRIFT_PPM: "5000"              # runs 5 ms fast every second
      PHASES: normal
    networks: [relojeria]

  client-c:
    profiles: [cristian]
    image: relojeria/cristian-client
    depends_on: [time-server, client-a]
    environment:
      <<: *cristian-client
      CLIENT_NAME: servidor-C
      CLOCK_OFFSET_SECONDS: "-5"           # 09:59:55
      PHASES: normal
    networks: [relojeria]

  # ---------------------------------------------------------------- Exercise 2: Berkeley
  nodo-1:
    profiles: [berkeley]
    build: { context: ., args: { PROJECT: Berkeley.Node } }
    image: relojeria/berkeley-node
    environment:
      <<: *common-env
      NODE_NAME: nodo-1
      CLOCK_OFFSET_SECONDS: ${NODE1_OFFSET:-10}
    ports: ["18081:8080"]
    networks: [relojeria]

  nodo-2:
    profiles: [berkeley]
    image: relojeria/berkeley-node
    depends_on: [nodo-1]
    environment:
      <<: *common-env
      NODE_NAME: nodo-2
      CLOCK_OFFSET_SECONDS: ${NODE2_OFFSET:--6}
      CLOCK_DRIFT_PPM: ${NODE2_DRIFT_PPM:-2000}
    ports: ["18082:8080"]
    networks: [relojeria]

  nodo-3:
    profiles: [berkeley]
    image: relojeria/berkeley-node
    depends_on: [nodo-1]
    environment:
      <<: *common-env
      NODE_NAME: nodo-3
      CLOCK_OFFSET_SECONDS: ${NODE3_OFFSET:-4}
    ports: ["18083:8080"]
    networks: [relojeria]

  coordinador:
    profiles: [berkeley]
    build: { context: ., args: { PROJECT: Berkeley.Coordinator } }
    image: relojeria/berkeley-coordinator
    depends_on: [nodo-1, nodo-2, nodo-3]
    environment:
      <<: *common-env
      COORDINATOR_NAME: coordinador
      NODES: http://nodo-1:8080,http://nodo-2:8080,http://nodo-3:8080
      CLOCK_OFFSET_SECONDS: "0"
      INTERVAL_SECONDS: ${BERKELEY_INTERVAL:-5}
      ROUNDS: ${BERKELEY_ROUNDS:-3}
      OUTLIER_THRESHOLD_SECONDS: ${OUTLIER_THRESHOLD:-0}
    networks: [relojeria]
```

```
  # ------------------------------------------------------------------ Exercise 3: Lamport
  orders-service:
    profiles: [lamport]
    build: { context: ., args: { PROJECT: Lamport.Orders } }
    image: relojeria/lamport-orders
    depends_on: [inventory-service, payments-service]
    environment:
      <<: *common-env
      CLOCK_OFFSET_SECONDS: "0"
      INVENTORY_URL: http://inventory-service:8080
      PAYMENTS_URL: http://payments-service:8080
    ports: ["8090:8080"]
    networks: [relojeria]

  inventory-service:
    profiles: [lamport]
    build: { context: ., args: { PROJECT: Lamport.Inventory } }
    image: relojeria/lamport-inventory
    environment:
      <<: *common-env
      CLOCK_OFFSET_SECONDS: "-3"           # its physical clock is 3 s behind
    networks: [relojeria]

  payments-service:
    profiles: [lamport]
    build: { context: ., args: { PROJECT: Lamport.Payments } }
    image: relojeria/lamport-payments
    environment:
      <<: *common-env
      CLOCK_OFFSET_SECONDS: "5"            # its physical clock is 5 s ahead
      PRELOAD_EVENTS: "4"                  # payments of other orders before this flow
    networks: [relojeria]

networks:
  relojeria:
    driver: bridge
```